목차

- Windows 앱 만들기
  - 데스크톱 앱 만들기
    - 개요
      - 앱 플랫폼 선택하기
      - 세팅하기
        - 장치에서 개발할 수 있도록 하기
        - 개발자 모드 기능 및 디버깅
        - 개발자 계정 생성
    - Windows 앱을 위한 Visual Studio 템플릿
    - 데스크 톱 앱 현대화하기
      - 개요
      - Windows 런타임 API 호출
        - 개요
        - Windows 런타임 API 사용 가능하게
      - WinRT XAML controls 호스팅(XAML Islands)
        - 개요
        - XWPF 와 Windows Forms apps 에서 XAML Islands
          - 표준 WinRT XAML control 호스트
          - 커스텀 WinRT XAML control 호스트
        - C++ Win32 앱에서 XAML Islands
          - WinRT XAML hosting API 사용
          - 표준 WinRT XAML control 호스트
          - 커스텀 WinRT XAML control 호스트
          - 응용 시나리오
      - Visual layer (Windows.UI.Composition) 사용하기
        - 개요
        - WPF
        - Windows Forms
        - Win32
      - 패키지 ID 가 필요한 기능
        - 개요
        - 패키지 확장자로 통합
        - 파일 탐색기로 패키지된 데스크톱 앱 통합

            - 패키지된 앱에 대한 활성화 정보 가져오기
            - UWP 구성 요소로 확장
            - 패키지된 앱 배포
        - 튜토리얼: WPF 앱 현대화하기
            - 개요
            - Contoso Expenses 앱을 .NET Core 3 로 마이그레이션
            - XAML Islands 를 이용하여 UWP 잉크캔버스 컨트롤 추가
            - XAML Islands 를 이용하여 UWP 캘린더뷰 컨트롤 추가
            - Windows 10 사용자 활동과 알림 추가
            - MSIX 로 패키지/배포
        - 패키징되지 않은 데스크톱 앱에 ID 부여
    - Windows UI 라이브러리(WinUI)
        - 개요
        - Windows UI Library 3 Preview 4
            - 개요 및 릴리스 노트
            - WinUI 3 으로 데스크톱 앱 만들기 시작하기
            - WinUI 3 으로 UWP 앱 만들기 시작하기
            - C++/WinRT 를 사용하는 WinUI 3 앱에 대한 XAML 컨트롤 템플릿
            - C#를 사용하는 WinUI 3 앱에 대한 XAML 컨트롤 템플릿
        - Windows UI 2.x 라이브러리
            - 개요
            - 셋업
                - WinUI 2.x 시작해 보기
                - NuGet 패키지 목록
            - 릴리스 노트
                - 개요
                - WinUI 2.5
                - WinUI 2.4
                - WinUI 2.3
                - WinUI 2.2
                - WinUI 2.1
                - WinUI 2.0
        - WinUI API 레퍼런스
    - Project Reunion 으로 Windows 앱 제작(사전 릴리스)

- 개요
- MRT Core
- DWriteCore

- Windows 10X 개발
  - Windows 10X 개발 FAQ
- Fluent Design System
- 접근성
- 스피치
- 기능 및 기술
- 최근 문서 업데이트
- Microsoft Build 2020 관련 문서

Windows PC 데스크톱 앱 개발하기

2/18/2021

이 자료에서는 Windows 용 데스크톱 앱을 만들거나 Windows 10 의 최신 환경을 적용하기 위해 기존 데스크톱 앱을 업데이트하는 데 필요한 정보를 제공합니다.

데스크톱 앱 플랫폼

Windows PC 용 데스크톱 앱을 구축하는 데는 네 가지 주요 플랫폼이 있습니다. 각 플랫폼은 앱의 라이프사이클을 정의하는 앱 모델, Word, Excel, Photoshop 과 같은 데스크톱 앱을 생성하고 Windows 기능을 사용하기 위해 관리되는 API 또는 기본 API 에 액세스할 수 있는 UI 프레임워크 및 UI 컨트롤 세트를 제공합니다.

추가적인 자료와 함께 세세한 비교를 보려면, 앱 플랫폼 선택하기 부분을 보십시오.

플랫폼	설명	문서 등 자료
Universal Windows Platform (UWP)	Windows 10 앱 및 게임을 위한 최첨단 플랫폼. UWP 컨트롤과 API 를 독점적으로 사용하는 UWP 앱을 구축하거나, 다른 플랫폼 중 하나를 사용하여 구축된 데스크톱 앱에서 UWP 컨트롤과 API 를 사용할 수 있습니다.	시작하기 API 레퍼런스 샘플
C++/Win32	Windows 및 하드웨어에 직접 액세스해야 하는 기본 Windows 앱의 플랫폼입니다.	시작하기 API 레퍼런스 샘플
WPF	XAML UI 모델을 사용하여 그래픽 기반으로 관리되는 Windows 애플리케이션을 위한 확립된 .NET 기반 플랫폼. .NET Core 3 또는 모든 .NET Framework 를 대상으로 할 수 있습니다.	시작하기 API 레퍼런스(.NET) 샘플
Windows Forms	경량 UI 모델로 관리되는 LOB(Line-of-Business) 애플리케이션용으로 설계된 .NET 기반 플랫폼. .NET Core 3 또는 모든 .NET Framework 를 대상으로 할 수 있습니다.	시작하기 API 레퍼런스(.NET)

향후 로드맵

앞으로 우리는 Windows UI Library(WinUI) 및 Project Reunion 을 통해 Windows 앱 개발 플랫폼을 발전시키고 있습니다.

- **WinUI** 는 Windows 10 앱용 기본 UX(사용자 환경) 프레임워크입니다. WinUI 는 Windows 10 의 하위 버전을 대상으로 하는 UWP 앱에 대한 WinRT 컨트롤의 새로운 버전과 업데이트된 버전을 제공하는 툴킷으로 시작했습니다. 현재 WinUI 3(현재 프리뷰)인 WinUI 는 UWP, .NET 및 Win32 앱 플랫폼 전반에 걸쳐 Windows 10 앱을 위한 UI(기본 사용자 인터페이스) 프레임워크로 성장하고 있습니다. 추가적인 정보는 Windows UI 라이브러리 (WinUI) 자료를 보십시오.
- **Project Reunion**(현재 프리뷰)은 Windows 앱 개발 플랫폼의 다음 진화를 나타내는 광범위한 새로운 개발자 구성 요소 및 도구의 태명입니다. Project Reunion 은 광범위한 대상 윈도우즈 10 OS 버전의 모든 앱에서 일관된 방식으로 사용할 수 있는 통합 API 및 툴 세트를 제공합니다. Project Reunion 은 UWP 및 네이티브 Win32, dotNET 과 같은 기존 Windows 앱 플랫폼과 프레임워크를 개발자들이 이러한 플랫폼 전반에 걸쳐 의존할 수 있는 공통 API 및 툴셋으로 보완합니다. 추가적인 정보는 Project Reunion 자료를 보십시오.

기존 데스크톱 앱을 Windows 10 용으로 업데이트

기존 WPF, Windows Forms 또는 기본 Win32 데스크톱 앱이 있는 경우 Windows 10 및 UWP(Universal Windows Platform)는 앱에서 최신 환경을 제공하는 데 사용할 수 있는 많은 기능을 제공합니다. 이러한 기능 중 대부분은 다른 플랫폼을 위해 앱을 다시 작성할 필요 없이 자신만의 속도로 앱에서 채택할 수 있는 모듈형 구성 요소로 제공됩니다.

기존 데스크톱 앱을 개선하는 데 사용할 수 있는 몇 가지 기능:

- MSIX 를 사용하여 데스크톱 앱을 패키징 및 배포합니다. MSIX 는 모든 Windows 앱에 범용 패키지 환경을 제공하는 최신 Windows 앱 패키지 형식입니다. MSIX 는 MSI, .appx, App-V 및 ClickOnce 설치 기술의 가장 좋은 측면을 결합하며, 안전하고 신뢰할 수 있도록 구축되었습니다.
- **패키지 확장**을 사용하여 데스크톱 앱을 Windows 10 과 통합할 수 있습니다. 예를 들어, 앱의 타일을 시작하거나, 앱을 공유 대상으로 만들거나, 앱에서 토스트 알림을 보냅니다.

  XAML Islands 를 사용하여 UWP XAML 컨트롤을 데스크톱 앱에서 호스팅합니다. 대부분의 최신 Windows 10 UI 기능은 UWP XAML 컨트롤에서만 사용할 수 있습니다.

추가적인 정보는 다음 자료를 보십시오.

자료	설명
데스크톱 앱 현대화하기	WPF, Windows Forms 및 C++ Win32 앱을 포함한 모든 데스크톱 앱에서 사용할 수 있는 최신 Windows 10 및 UWP 개발 기능에 대해 설명합니다.
튜토리얼: WPF 앱 현대화하기	UWP 잉크 및 캘린더 컨트롤을 앱에 추가하고 MSIX 패키지에 패키징하여 기존 WPF LOB(Line-of-Business) 샘플 앱을 현대화하려면 단계별 지침을 따르십시오.

새 데스크톱 앱 만들기

새 Windows 앱을 만들고자 한다면 다음 자료가 시작하는 데 도움이 될 수 있습니다.

자료	설명
앱 플랫폼 선택하기	주요 데스크톱 앱 플랫폼을 심층적으로 비교하고 사용자의 요구에 적합한 플랫폼을 선택하는 데 도움이 됩니다. 이 문서에서는 각 플랫폼에 대한 문서에 대한 유용한 링크도 제공합니다.
Windows 앱을 위한 Visual Studio 프로젝트 템플릿	Visual Studio 에서 C# 또는 C++를 사용하여 Windows 10 장치용 앱을 만드는 데 도움이 되는 프로젝트 및 항목 템플릿을 설명합니다.
데스크톱 앱 현대화하기	WPF, Windows Forms 및 C++ Win32 앱을 포함한 모든 데스크톱 앱에서 사용할 수 있는 최신 Windows 10 및 UWP 개발 기능에 대해 설명합니다.
기능 및 기술	각 기본 데스크톱 앱 플랫폼과 관련 문서에 대한 링크를 통해 액세스할 수 있는 Windows 기능에 대한 개요를 제공합니다.

관련 문서 및 기술

자료	설명
.NET Core 3.1	WPF/Windows Forms 앱을 위한 개선을 비롯한 .NET Core 3.1 의 최신 기능에 대하여 설명합니다.
.NET 5	이 자료에는 3.1 의 .NET Core 다음 릴리스인 .NET 5 에 포함된 내용이 자세히 나와 있습니다.
Desktop guide for WPF and .NET Core	모든 .NET 프레임워크가 아닌 .NET Core 대상 WPF 앱 개발
Azure	Azure 클라우드 서비스를 통해 애플리케이션의 범위를 확장하십시오.
Visual Studio	앱 및 서비스를 개발하기 위해 Visual Studio 사용법을 배우십시오.
MSIX	모든 Windows 앱을 최신 범용 패키지 형식으로 패키징 및 배포합니다.
Windows AI	Windows AI 를 사용하여 앱의 복잡한 문제에 대한 지능형 솔루션을 구축합니다.
Windows Containers	완전히 격리된 빠른 윈도우즈 환경에서 애플리케이션을 종속성으로 패키지화하십시오.
Progressive Web Apps	웹 앱을 Windows 10 에서 UWP 앱으로 배포하고 실행할 수 있는 진행형 웹 앱으로 변환합니다.
Xamarin	.NET 코드 및 플랫폼별 사용자 인터페이스를 사용하여 Windows, Android, iOS 및 macOS 용 크로스 플랫폼 앱을 구축합니다.
Docs archive for Windows 8.x and earlier	Windows 8.x 및 이전 버전의 앱 작성에 대한 보관된 문서에 액세스합니다.

Windows 앱 플랫폼 선택하기

2/18/2021

Windows PC 용 새 데스크톱 응용 프로그램을 만들려는 경우 가장 먼저 사용할 응용 프로그램 플랫폼을 결정합니다. Windows(윈도우)는 4 가지 주요 애플리케이션 플랫폼을 제공하며 각각 다른 강점을 가지고 있습니다.

- Universal Windows Platform (UWP): 이 플랫폼은 Windows 10 을 실행하는 모든 장치에 공통 유형 시스템, API 및 애플리케이션 모델을 제공합니다. 네이티브 또는 관리 가능 프로그램이 있습니다.
- WPF and Windows Forms: 이러한 .NET 기반 플랫폼은 중앙 관리 애플리케이션을 위한 공통 유형 시스템, API 및 애플리케이션 모델을 제공합니다.
- Win32: 이 플랫폼은 Windows 및 하드웨어에 직접 액세스해야 하는 기본 C/C++ Windows 애플리케이션을 위한 원래 플랫폼입니다. 따라서 Win32 API 는 최고 수준의 성능과 시스템 하드웨어에 직접 액세스해야 하는 애플리케이션을 위한 선택 플랫폼으로 자리 잡게 됩니다.

각 플랫폼에는 완벽한 UI 프레임워크와 Word, Excel 및 Photoshop 과 같은 데스크톱 앱을 생성하여 기존 Windows 데스크톱에서 실행되며 해당 환경의 특정 기능을 최대한 활용할 수 있는 UI 컨트롤 세트가 포함되어 있습니다. Windows 10 에서는 각 플랫폼에서도 Windows UI 라이브러리를 사용하여 사용자 인터페이스를 만들 수 있습니다.

이러한 플랫폼 중 일부는 일부 특성을 공유하며 특정 유형의 애플리케이션에 더 적합합니다. 예를 들어 UWP 와 .NET 은 Visual Studio 와 긴밀하게 통합됩니다. 이는 특히 개발자 생산성, 정교하고 사용자 정의가 가능한 UI 및 애플리케이션 보안 분야에서 많은 이점을 제공합니다. 이러한 프레임워크는 UI 를 빠르게 생성하는 시각 디자이너와 UI 마크업을 지원하기 때문에 특히 LOB(Line-of-Business) 애플리케이션에 적합합니다.

> **NOTE**
> 어떤 앱 플랫폼을 선택하든 많은 Windows 10 기능을 사용하여 앱에서 현대적인 경험을 제공할 수 있습니다. 예를 들어 WPF, Windows Forms 또는 Win32 API 를 사용하여 데스크톱 앱을 빌드한 경우에도 MSIX 패키지 배포를 계속 사용할 수 있습니다. 데스크톱 앱을 현대화하는 모든 방법에 대한 자세한 내용은 데스크톱 앱 현대화를 참조하십시오.

UWP

UWP 는 Windows 10 응용 프로그램과 게임을 위한 최첨단 플랫폼입니다. XAML 마크업을 사용하여 UI 와 코드(비즈니스 논리)를 구분하는 사용자 지정이 가능한 플랫폼입니다. UWP 는 정교한 UI, 스타일 사용자 지정 및 그래픽 집약적인 시나리오가 필요한 데스크톱 애플리케이션에 적합합니다. 또한 UWP 는 기본 UX 환경을 위한 Fluent Design System 을 기본적으로 지원하며 Windows Runtime(WinRT) API 에 대한 액세스를 제공합니다. Fluent 를 채택함으로써 UWP 는 잉크, 터치, 게임패드, 키보드, 마우스와 같은 일반적인 입력 방식을 자동으로 지원합니다.

UWP 를 사용하여 Windows PC 용 데스크톱 응용 프로그램을 만들 수 있을 뿐만 아니라 Xbox, HoloLens 및 Surface Hub 응용 프로그램에서 유일하게 지원되는 플랫폼입니다. UWP 는 최신 첨단 애플리케이션 플랫폼입니다.

추가적인 정보는 다음 자료들에 있습니다.

- 시작하기
- 프로젝트 템플릿
- 디자인과 UI
- 기술 및 기능
- API 레퍼런스
- 샘플

WPF

WPF 는 .NET Core 또는 full .NET Framework 에 액세스할 수 있는 관리되는 Windows 애플리케이션을 위한 플랫폼이며 XAML 마크업을 사용하여 UI 와 코드를 구분한다. 이 플랫폼은 정교한 UI, 스타일 사용자 지정 및 그래픽

집약적인 시나리오가 필요한 데스크톱 애플리케이션을 위해 설계되었습니다. WPF 개발 기술은 UWP 개발 기술과 유사하므로 WPF 에서 UWP 앱으로 마이그레이션하는 것이 Windows Forms 에서 마이그레이션하는 것보다 쉽습니다.

추가적인 정보는 다음 자료들에 있습니다.

- 시작하기(WPF)
- 프로젝트 템플릿
- 첫 앱 만들기 (.NET Core)
- 첫 앱 만들기 (.NET Framework)
- WPF 앱을 .NET Core 로 마이그레이션
- API 레퍼런스(.NET)
- 샘플

Windows Forms

Windows Forms 는 .NET Core 또는 full .NET 프레임워크에 대한 액세스와 가벼운 UI 모델이 있는 관리되는 Windows 응용 프로그램을 위한 원래 플랫폼입니다. 개발자가 플랫폼을 처음 사용하는 개발자에게도 애플리케이션 구축을 신속하게 시작할 수 있도록 지원하는 데 탁월합니다. 이 플랫폼은 시각적 및 비시각적 드래그 앤 드롭 제어 기능이 내장된 폼 기반 신속한 애플리케이션 개발 플랫폼입니다. Windows Forms 는 XAML 을 사용하지 않으므로 나중에 응용 프로그램을 UWP 로 확장하기로 결정하려면 UI 를 완전히 다시 써야 합니다.

추가적인 정보는 다음 자료들에 있습니다.

- Windows Forms 시작하기
- 프로젝트 템플릿
- 첫 Windows Forms 앱 만들기
- 튜토리얼: 사진 뷰어 만들기
- API 레퍼런스 (.NET)
- Windows Forms 앱 개선하기

Win32

Win32 API 를 C++와 함께 사용하면 WinRT 와 .NET 과 같은 관리 런타임 환경에서 가능한 것보다 관리되지 않은 코드로 대상 플랫폼을 더 많이 제어함으로써 최고 수준의 성능과 효율성을 달성할 수 있습니다. 그러나 애플리케이션 실행을 이러한 수준으로 제어하려면 올바르게 하려면 더 많은 주의를 기울여야 하며, 개발 생산성이 대체로 낮습니다.

다음은 고성능 애플리케이션을 구축할 수 있도록 Win32 API 및 C++가 제공하는 기능에 대한 몇 가지 강조 사항입니다.

- 리소스 할당, 개체 수명, 데이터 레이아웃, 정렬, 바이트 패킹 등에 대한 엄격한 제어를 포함한 하드웨어 수준 최적화
- 내장 기능을 통해 SSE 및 AVX 와 같은 성능 지향 명령 집합에 액세스
- 템플릿을 이용한 효율적이고 안전한 제네릭 프로그래밍
- 효율적이고 안전한 컨테이너/알고리즘
- DirectX, 특히 Direct3D 및 DirectCompute(UWP 는 DirectX interop 도 제공.)

추가적인 정보는 다음 자료들에 있습니다.For more information, see the following articles:

- 시작하기
- 프로젝트 템플릿
- 첫 Win32/C++ 앱 만들기
- 기술 및 기능
- API 레퍼런스
- 샘플

플랫폼 비교: UWP, WPF, and Windows Forms

다음 표는 위 플랫폼에 대한 더 자세한 비교를 보여줍니다.

기능/시나리오	UWP	WPF	WINDOWS FORMS
지원 버전	Windows 10	Windows 7 이후	Windows 7 이후
언어	C#, C++/WinRT, C++/CX, VB, JavaScript	C#, C++/CLI (C++ 확장), F#, VB	C#, C++/CLI (C++ 확장), F#, VB
UI 런타임	Native (C++/WinRT and C++/CX) and managed (.NET Native)	Managed (.NET Framework and .NET Core 3)	Managed (.NET Framework and .NET Core 3)
오픈 소스	Yes (Windows UI Library only)	Yes (.NET Core only)	Yes (.NET Core only)
XAML 지원	Yes	Yes	No
강점	- XAML markup for UI - 풍부한 커스텀 UX - 기존 코드 기반이 .NET 표준과 호환 - 고 DPI 지원 - Windows 장치 간 다중 입력 유형 지원(터치, 펜, 게임패드, 마우스, 키보드 등) - Xbox, HoloLens, IoT or Surface Hub 지원 - 선택적 밀집 UI - 네이티브 C++ 지원 - 배터리 수명 최적화 - 최신 접근성 지원(읽어주기 등) - 많은 유용한 텍스트 데이터 (빌트인 스펠체크 등) - 잉크 그림 지원 - 애플리케이션 컨테이너를 통한 안전한 실행(ex: 신뢰할 수 없는 컨텐츠는 안전모드)	- XAML markup for UI - 풍부한 커스텀 UX - 마이크로소프트 및 파트너의 대규모 제어 기능 모음 - 밀집 UI - Windows 7 지원 - 입력 확인을 위한 플랫폼 지원	- 빠른 개발 - UI 개발을 위한 WYSIWYG 에디터 - 마이크로소프트 및 파트너의 대규모 제어 기능 모음 - 밀집 UI - Windows 7 지원 - 키보드/마우스 입력

지원이 제한된 시나리오	- 다중 창 지원 [1] - 입력 확인을 위한 플랫폼 지원 [1] - Windows 7 미지원 - 몇몇 Windows 런타임 API 가 Windows 10 의 특정 버전만 지원함 - 플랫폼 완전 지원과 셸 통합(예를 들어 UWP 는 현재 시스템 트레이 통합 또는 모든 장치에 대한 전체 액세스를 지원하지 않음) - 디스크의 모든 파일 직접 접근 - ADO.NET - 비.NET 표준 또는 비 Windows App 인증서 Kit 호환 API 를 사용하는 기존 코드베이스 클래스 라이브러리 - 지역 네트워크 루프백 지원(즉, 대상 디바이스에 루프백 예외를 만들지 않고 앱이 localhost 와 통신해야 하는 경우) - 강력한 파일 입출력	- 고 DPI 지원 [2] - 터치 입력 [2]	- 그래픽/UX(터치/애니메이션 등) - 고 DPI 지원 [2] - 터치 입력 [2] - 뷰/데이터 모델의 추상화 수 - 커스텀 가능 UI

[1] 향후 Windows 10 릴리스에서 이 시나리오를 다룰 기능을 공개적으로 발표했습니다.

[2] 플랫폼에는 이 시나리오에 대한 1 급 API 지원이 부족하지만 개발자는 해결 방법을 통해 이 시나리오를 지원할 수 있습니다.

Windows UI 라이브러리를 Windows 앱에 사용하기

기본 Windows 앱 플랫폼을 보완하기 위해 앱에서 Windows UI 라이브러리를 사용할 수도 있습니다. WinUI 는 Windows 10 의 하위 버전을 대상으로 하는 UWP 앱에 대한 WinRT 컨트롤의 새로운 버전과 업데이트된 버전을 제공하는 툴킷으로 시작했습니다. 현재 윈 UI 3(프리뷰) WinUI 는 UWP, .NET 및 Win32 앱 플랫폼 전반에 걸쳐 Windows 10 앱을 위한 UI(기본 사용자 인터페이스) 프레임워크로 성장하고 있습니다. 다음과 같은 방법으로 사용할 수 있습니다.

- WinUI 2.x:
  - UWP 앱은 Windows SDK 에서 제공하는 WinRT 컨트롤 대신 WinUI 2.x 컨트롤을 사용할 수 있습니다. 이 WinUI 릴리스에는 Windows SDK 의 새 컨트롤과 기존 컨트롤의 업데이트된 버전이 모두 포함되어 있습니다.
  - XAML Islands 를 사용하여 WinUI 2.x 컨트롤을 호스팅하도록 기존 WPF, Windows Forms 및 C++/Win32 앱을 업데이트할 수 있습니다.
- WinUI 3 (프리뷰):

WinUI 3 부터는 완전히 WinUI 기반 UI 를 사용하는 dotNET 및 C++/Win32 앱과 UWP 앱을 생성할 수 있습니다. 이 릴리스에는 이러한 앱을 만드는 데 필요한 모든 것을 제공하는 Visual Studio 프로젝트 템플릿이 포함되어 있습니다.

> **NOTE**
> 3 판은 아직 프리뷰이며, 현재 생산성 앱에 사용할 수 없습니다.

다른 앱 플랫폼

Progressive Web Apps (PWAs)

개발자들은 PWA 를 통해 웹 사이트 코드를 패키징하여 Windows 10 PC 에 애플리케이션처럼 설치 및 실행할 수 있습니다. 자세한 내용은 여기를 참조하십시오.

Xamarin

Xamarin 을 사용하여 iOS 및 Android 에서도 실행할 수 있는 Windows 10 용 크로스 플랫폼 애플리케이션을 구축할 수 있습니다. 자세한 내용은 여기를 참조하십시오.

Uno Platform

Uno 플랫폼은 Windows UWP 기반 코드(C# 및 XAML)를 iOS, Android, MacOS, Linux 및 Web Assembly 에서 실행할 수 있게 합니다. 윈도우 10 2004(19041)의 UWP 에 대한 전체 API 정의와 UWP API 의 일부를 구현(Windows.UI.Xaml 등)하여 UWP 애플리케이션이 이러한 플랫폼에서 실행될 수 있도록 한다. 자세한 내용은 Uno 플랫폼 문서를 참조하십시오.

세팅하기

2/18/2021

Windows 10 개발에 입문하신 것을 환영합니다! 다음 안내를 따라 WinRT API(aka UWP)를 이용해 Windows 10 용 앱을 만들어 보세요.

Visual Studio 다운로드/업데이트

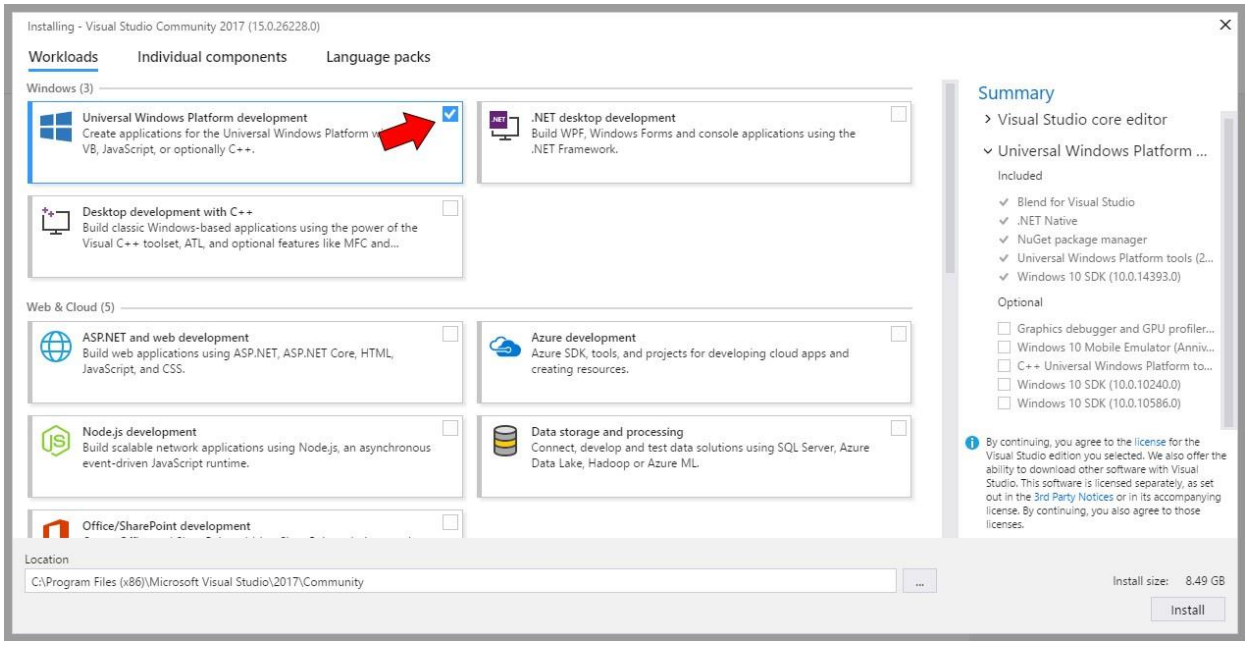
> **NOTE**
> 새 버전의 Visual Studio 는 다음 사진과는 다르게 생겼을 수 있습니다. 걱정하지 마세요! 그렇다 해도 다음 안내를 따르면 쉽게 설치할 수 있습니다.

Microsoft Visual Studio 2019 는 당신이 앱을 설계하고, 코드를 쓰고, 테스트하고, 디버깅하는 것을 도와줍니다.

지금 Visual Studio 2019 가 없다면, 무료인 Microsoft Visual Studio Community 2019 를 설치하실 수 있습니다. 이 다운로드에는 앱 테스트를 위한 장치 시뮬레이터가 포함됩니다.

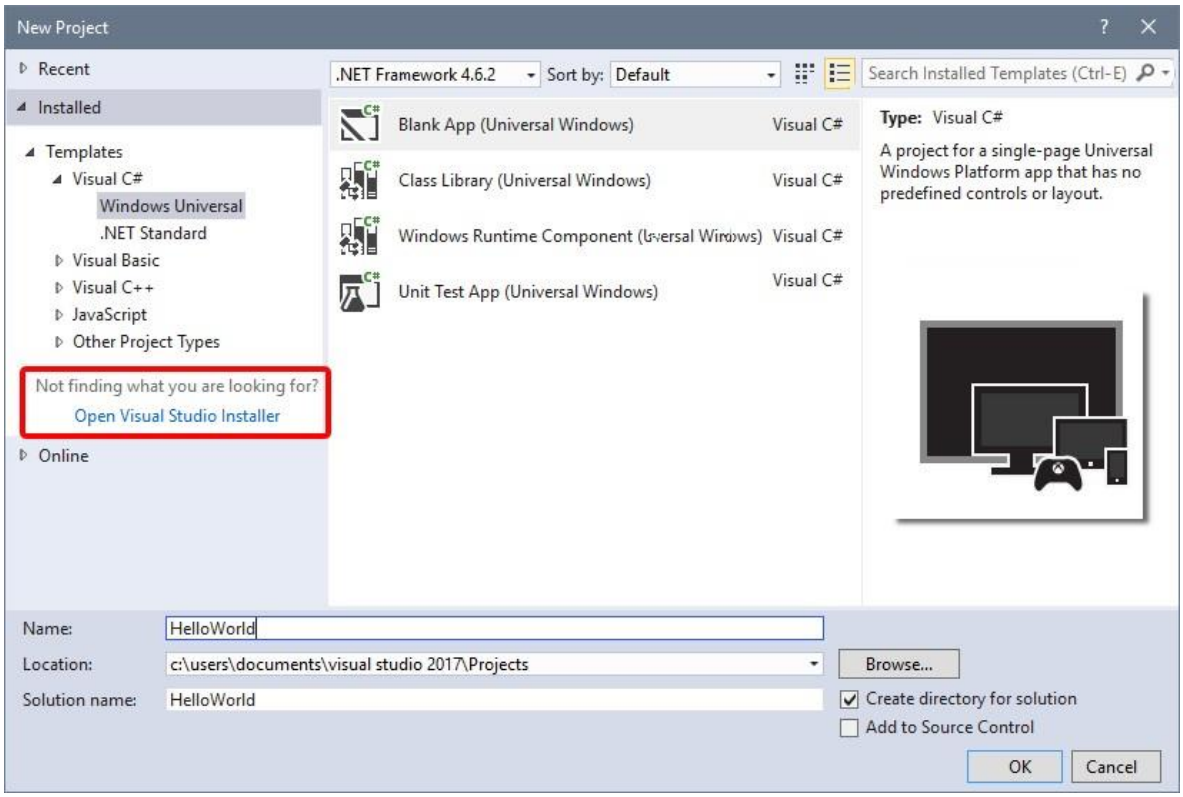
- Windows 10 개발자 도구 다운로드

설치할 때, 다음과 같이 '유니버설 Windows 플랫폼 개발'에 체크해 주세요.



비주얼 스튜디오 사용에 대한 안내는 여기를 참고해 주세요. 비주얼 스튜디오 시작하기

기존에 이미 비주얼 스튜디오를 사용하고 있었지만 구성 요소 몇 가지를 가지고 있지 않다면, '새 프로젝트' 대화 상자에서 위와 같은 설치 프로그램을 다시 열 수 있습니다.



장치에서 개발할 수 있도록 하기

실제 PC 나 스마트폰에서 UWP 앱을 테스트하는 것은 중요합니다. 만든 앱을 배포하기 전에 개발을 가능하게 해야 합니다.

- 자세한 설명은 장치에서 개발할 수 있도록 하기 부분을 보십시오.

앱 개발자로 등록하기

지금 개발을 시작할 수도 있지만, 스토어에 게시하려면 개발자 계정이 필요합니다.

- 개발자 계정을 만드려면 가입 페이지로 이동해 주세요.

다음은?

필요한 도구와 개발 옵션을 활성화하였다면, 그 다음은 시작에 대한 안내를 보십시오.

- 앱 플랫폼 선택하기
- 코딩 시작하기

Windows 개발 셋업 스크립트

당신의 디바이스에서 다른 기능이나 패키지를 설치, 혹은 커스터마이징을 원하신다면 시스템 세팅 스크립트를 보세요.

- 개발자 셋업 스크립트

더 많은 도구를 알아보고 싶으세요?

모든 도구 및 그 다운로드는 이곳에서 찾을 수 있습니다.

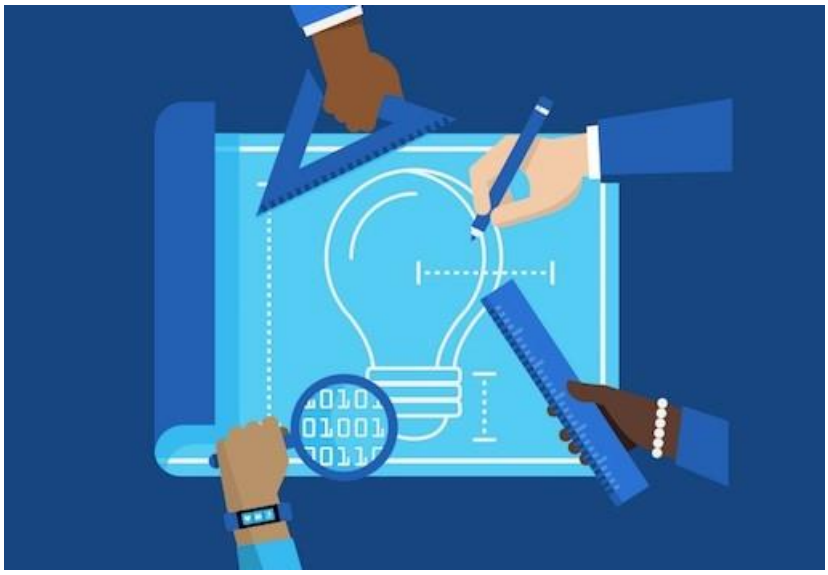
같이 보기

- UWP 개발자를 위한 코드 샘플
- UWP 앱이란?
- Windows 가입하기

장치에서 개발할 수 있도록 하기

11/2/2020

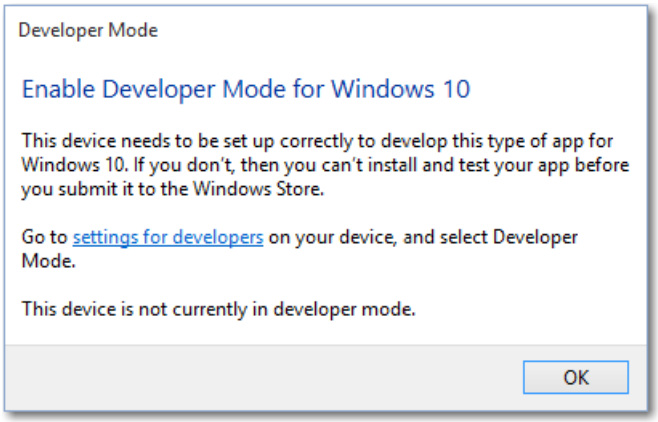
개발자 모드 활성화, 앱 사이드 로드하기, 다른 개발자 모드 사용하기



> **IMPORTANT** 당신의 PC 에서 앱을 만드는 것이 아니라면 개발자 모드를 활성화시킬 필요는 없습니다. 컴퓨터의 문제를 해결하려고 한다면, Windows 도움말을 확인해 보세요. 개발이 처음이라면 필요한 도구를 받아 보세요.

게임, 웹 검색, 전자 메일 또는 Office 앱과 같은 일상적인 작업에 컴퓨터를 사용하는 경우 개발자 모드를 활성화할 필요가 없으며 실제로 활성화해서는 안 됩니다. 이 페이지의 나머지 정보는 여러분에게 중요하지 않을 것이고, 여러분은 안전하게 여러분이 하던 일을 다시 할 수 있습니다. 이 한글화 문서를 보는 경우 그럴 리는 없겠지만..

그러나 컴퓨터에서 Visual Studio 를 사용하여 소프트웨어를 처음 작성하는 경우 개발 PC 와 코드를 테스트하는 데 사용할 장치에서 개발자 모드를 활성화해야 합니다. 개발자 모드가 활성화되지 않은 상태에서 UWP 프로젝트를 열면 개발자 설정 페이지가 열리거나 이 대화 상자가 Visual Studio 에 나타납니다.



여기에서는, settings for developers 를 눌러 개발자 설정 페이지를 열어 주세요.

> NOTE
>
> 언제든지 개발자용 페이지로 이동하여 개발자 모드를 활성화하거나 비활성화할 수 있습니다. 작업 표시줄의 Cortana 검색 상자에 "개발자 설정"을 입력하면 됩니다.

개발자 설정

개발자 모드를 활성화하거나 다른 설정을 하려면,

1. 개발자 설정 대화 상자에서, 필요한 수준의 접근 권한을 선택하세요.
2. 주의 사항을 읽고 나서, '예'를 클릭해 주세요.

> NOTE
>
> 개발자 모드를 활성화하려면 관리자 권한이 필요합니다. 장치가 조직 소유인 경우, 이 옵션을 사용할 수 없을 수 있습니다.

개발자 모드 기능

개발자 모드는 개발자 라이센스에 대한 윈도우즈 8.1 요구 사항을 대체합니다. 개발자 모드 설정은 사이드로드 외에도 디버깅 및 추가 배포 옵션을 활성화합니다. 여기에는 이 장치를 배포할 수 있는 SSH 서비스 시작이 포함됩니다. 이 서비스를 중지하려면 개발자 모드를 비활성화해야 합니다.

데스크톱에서 개발자 모드를 활성화하면, 다음을 포함하는 패키지가 설치됩니다.

- Windows 장치 포털. 장치 포털 옵션이 켜진 경우에만 장치 포털이 활성화되며 방화벽 규칙이 구성됩니다.
- 앱 원격 설치를 허용하는 SSH 서비스에 대한 방화벽 규칙을 설치하고 구성합니다. 장치 검색을 활성화하면 SSH 서버가 켜집니다.

위 기능에 대한 자세한 설명이 보고 싶거나 설치 과정에 어려움이 발생한 경우 개발자 모드 기능 및 디버깅 쪽을 확인하십시오.

같이 보기

- Windows 가입하기
- 개발자 모드 기능 및 디버깅

개발자 모드 기능 및 디버깅

2/18/2021

앱에 개발자 모드를 설치하는 기본 사항에 관심이 있는 경우, 장치에서 개발할 수 있도록 하기에 설명된 지침을 따르십시오. 이 자료에서는 개발자 모드의 고급 기능, 이전 버전의 Windows 10 의 개발자 모드 및 개발자 모드 설치의 디버깅 실패를 다룹니다.

개발자 모드의 다른 기능

각 장치 제품군에 대해 추가 개발자 기능을 사용할 수 있습니다. 이러한 기능은 장치에서 개발자 모드를 사용하도록 설정한 경우에만 사용할 수 있으며 OS 버전에 따라 다를 수 있습니다.

다음은 Winodws 10 의 개발자 모드 기능입니다.

장치 포털

자세한 설명은 Windows 장치 포털 개요를 참고하세요.

장치별 설정 안내는 다음을 보시면 됩니다.

- 데스크톱 장치 포털
- HoloLens 장치 포털
- IoT 장치 포털
- 모바일 장치 포털
- Xbox 장치 포털

개발자 모드 또는 장치 포털을 사용하는 데 문제가 발생하면 알려진 문제 포럼을 참조하여 이러한 문제에 대한 해결 방법을 찾거나, 아래의 '개발자 모드 설치 실패' 부분을 보고 추가 세부 정보와 개발자 모드 패키지 차단 해제를 위해 어떤 WSUS KB 를 허용해야 하는지 알아보십시오.

앱 사이드로드

> **IMPORTANT** 최신 Windows 10 업데이트에서는 사이드로드가 기본적으로 설정되어 있으므로 이 설정은 표시되지 않습니다. 이전 버전의 Windows 10 인 경우 기본 설정에서는 Microsoft Store 에서만 앱을 실행할 수 있으며 Microsoft 가 아닌 소스에서 앱을 설치하려면 사이드 로딩을 사용하도록 설정해야 합니다.

앱 사이드로드 설정은 일반적으로 Microsoft Store 를 거치지 않고 관리되는 기기에 사용자 지정 앱을 설치해야 하는 회사나 학교 또는 Microsoft 가 아닌 다른 소스에서 앱을 실행해야 하는 모든 사용자가 사용합니다. 이 경우 조직은 설정 페이지의 이미지에 앞서 표시된 것처럼 UWP 앱 설정을 실행 중지하는 정책을 시행하는 것이 일반적입니다. 조직은 또한 앱의 사이드로드에 필요한 인증서와 설치 위치도 제공합니다. 자세한 내용은 TechNet 의 Windows 10 사이드로드 앱 및 Microsoft Intune 기본 문서 참조하십시오.

장치 계열에 따른 정보

- 데스크톱 계열: 패키지로 생성된 Windows PowerShell 스크립트("AddAppDevPackage.ps1")를 실행하여 애플리케이션 패키지(.appx) 및 애플리케이션 실행에 필요한 인증서를 설치할 수 있습니다. 자세한 내용은 UWP 앱 패키징을 참조하십시오.
- 모바일 계열: 필요한 인증서가 이미 설치되어 있는 경우 파일을 눌러 이메일 또는 SD 카드를 통해 사용자에게 전송된 .appx 를 설치할 수 있습니다.

사이드로드 앱은 신뢰 가능한 인증서가 없는 앱은 설치할 수 없으므로 개발자 모드보다 안전한 옵션입니다.

> **NOTE**
>
> 사이드로드를 하는 경우에도 신뢰할 수 있는 출처의 앱만 설치해야 합니다. Microsoft Store 에서 인증받지 않은 사이드로드 앱을 설치하는 경우, 당신은 앱의 사이드로드에 필요한 모든 권한을 얻었으며 앱 설치 및 실행으로 인한 모든 손상에 대해 전적으로 사용자에게 책임이 있다는 데 동의하는 것입니다. 개인 정보 처리 방침의 Windows -> Microsoft Store 섹션을 참조하세요.

SSH

장치에서 장치 검색 켜기를 설정하면 SSH 서비스가 활성화됩니다. 장치가 UWP 응용 프로그램의 원격 배포 대상인 경우에 사용됩니다. 서비스 이름은 'SSH 서버 브로커' 및 'SSH 서버 프록시'입니다.

> **NOTE**
>
> 이것은 Microsoft 의 OpenSSH 구현이 아닙니다.

SSH 서비스를 이용하기 위해 장치 검색을 활성화하여 핀 페어링을 허용할 수 있습니다. 다른 SSH 서비스를 실행하려면 다른 포트에 이 서비스를 설정하거나 개발자 모드 SSH 서비스를 해제할 수 있습니다. SSH 서비스를 끄려면 장치 검색을 끄십시오.

SSH 로그인은 "DevToolsUser" 계정을 통해 수행되며, 이 계정을 통해 인증에 대한 암호를 수락합니다. 이 암호는 장치 검색 "페어" 버튼을 누른 후 장치에 표시되는 PIN 이며 PIN 이 표시되는 동안에만 유효합니다. 또한 SFTP 하위 시스템은 Visual Studio 에서 느슨한 파일 배포가 설치된 DevelopmentFiles 폴더의 수동 관리를 위해 사용할 수 있습니다.

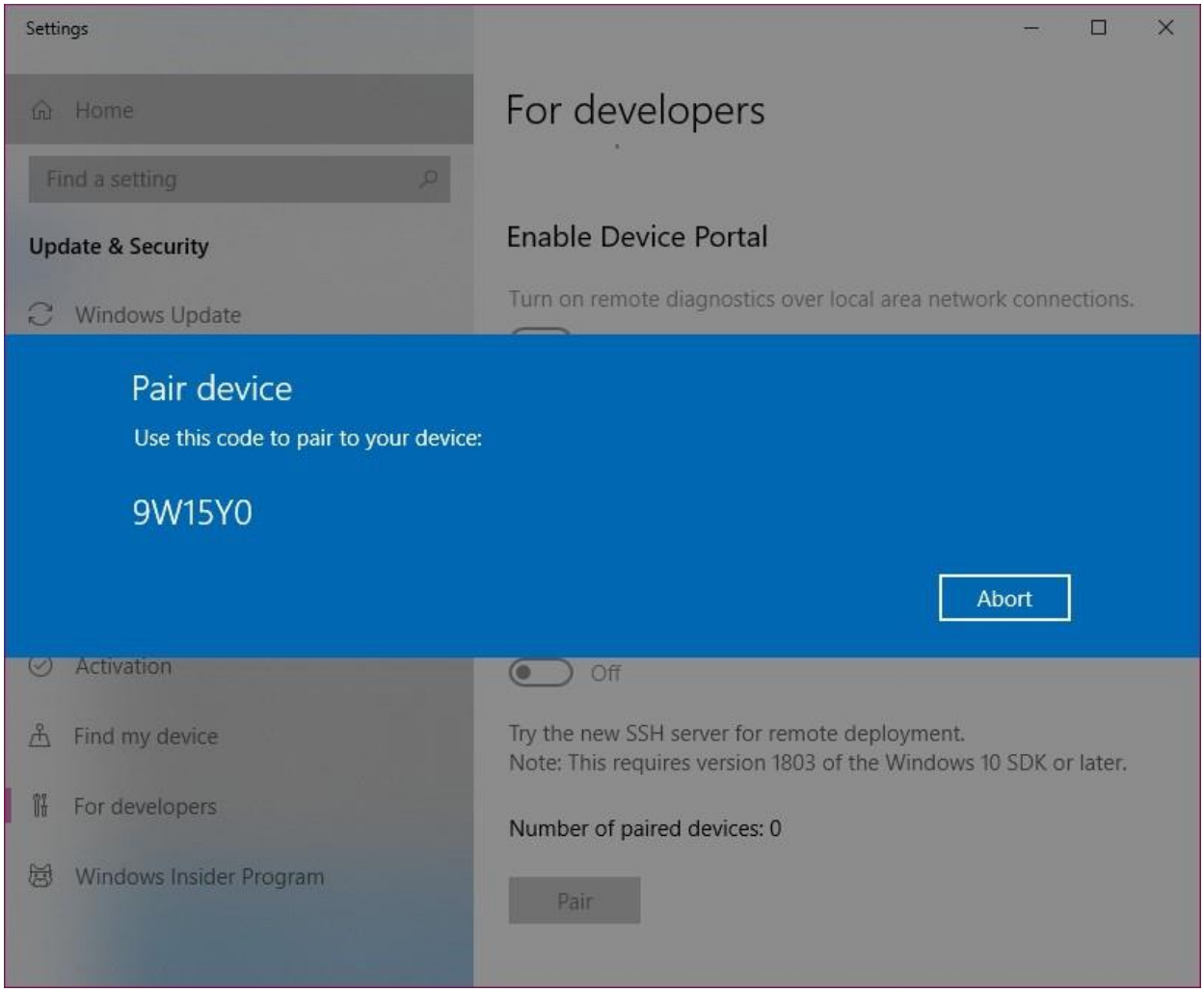
SSH 사용에 대한 주의 사항

Windows 에서 사용되는 기존 SSH 서버는 아직 프로토콜 규정을 준수하지 않으므로 SFTP 또는 SSH 클라이언트를 사용하려면 특별한 구성이 필요할 수 있습니다. 특히 SFTP 하위 시스템은 버전 3 이하에서 실행되므로 모든 연결 클라이언트가 이전 서버를 사용하도록 구성되어야 합니다. 이전 장치의 SSH 서버는 공개 키 인증에 ssh-ds 를 사용하며, OpenSSH 는 이를 더 이상 사용하지 않습니다. 이러한 장치에 연결하려면 SSH 클라이언트를 수동으로 구성하여 ssh-dss 를 수락해야 합니다.

장치 검색

장치 검색을 사용하도록 설정하면 mDNS 를 통해 네트워크의 다른 장치에 장치를 표시할 수 있습니다. 이 기능을 사용하면 장치 검색이 활성화되면 "페어" 버튼을 눌러 이 장치와 페어링하기 위한 SSH PIN 을 가져올 수도

있습니다. 장치를 대상으로 한 첫 번째 Visual Studio 배포를 완료하려면 이 PIN 프롬프트가 화면에 표시되어야 합니다.



장치를 배포 대상으로 만들려는 경우에만 장치 검색을 활성화해야 합니다. 예를 들어 장치 포털을 사용하여 테스트하기 위해 스마트폰에 앱을 배포하는 경우 개발 PC 가 아닌 스마트폰에서 장치 검색을 사용하도록 설정해야 합니다.

Windows 탐색기, 원격 데스크톱, PoerShell 을 위한 최적화(데스크톱)

데스크톱 장치 제품군에서 개발자 설정 페이지에는 개발 작업에 대해 PC 를 최적화하는 데 사용할 수 있는 설정에 대한 바로 가기가 있습니다. 각 설정에 대해 확인란을 선택하고 적용을 클릭하거나 설정 표시 링크를 클릭하여 해당 옵션에 대한 설정 페이지를 열 수 있습니다.

Notes

이전 버전의 Windows 10 Mobile 에서는 개발자 설정 메뉴에 크래시 덤프 옵션이 있었습니다. USB 를 통해서만 사용할 수 있는 것이 아니라 원격으로 사용할 수 있도록 장치 포털로 이동되었습니다.

Windows 10 PC 에서 Windows 10 장치로 앱을 배포하는 데 사용할 수 있는 도구는 여러 가지가 있습니다. 두 장치는 모두 유선 또는 무선 연결을 통해 네트워크의 동일한 서브넷에 연결되거나 USB 로 연결되어야 합니다. 나열된 두 가지 방법 모두 앱 패키지만 설치합니다.

- Windows 10 애플리케이션 배포(WinAppDeployCmd) 도구를 사용합니다. WinAppDeployCmd 도구에 대해 자세히 알아보십시오.
- 장치 포털을 사용하여 브라우저에서 Windows 10, 버전 1511 이상을 실행하는 모바일 장치로 배포할 수 있습니다. 장치 포털의 앱 페이지를 사용하여 앱 패키지(.appx)를 업로드하고 장치에 설치합니다.

개발자 모드 패키지 설치에 실패한 경우

네트워크 또는 관리 문제로 인해 개발자 모드가 제대로 설치되지 않는 경우가 있습니다. 개발자 모드 패키지는 브라우저의 장치 포털 또는 장치 검색을 사용하여 이 PC 에 원격으로 배포하려면 필요하지만 로컬 개발에는 배포되지 않습니다. 이러한 문제가 발생하더라도 Visual Studio 를 사용하거나 이 장치에서 다른 장치로 앱을 로컬로 배포할 수 있습니다.
이러한 문제에 대한 해결 방법 등을 알아보려면 알려진 이슈 포럼을 참조하세요.

> NOTE
> 개발자 모드가 올바르게 설치되지 않은 경우 피드백 요청을 제출하는 것이 좋습니다. 피드백 허브 앱에서 새 피드백 추가를 선택하고 개발자 플랫폼 범주 및 개발자 모드 하위 범주를 선택합니다. 피드백을 제출하면 Microsoft 에서 발생한 문제를 해결하는 데 도움이 됩니다.

패키지를 찾지 못함

"개발자 모드 패키지를 Windows Update 에서 찾을 수 없습니다. 에러 코드 0x80004005 자세히 보기" 이 오류는

네트워크 연결 문제, 엔터프라이즈 설정 또는 패키지가 누락되어 발생할 수 있습니다.

해결 방법:

1. 컴퓨터가 인터넷에 연결되어 있어야 합니다.
2. 도메인에 가입된 컴퓨터를 사용하는 경우 네트워크 관리자에게 문의하십시오. 모든 주문형 기능과 마찬가지로 개발자 모드 패키지는 WSUS 2.1 에서 기본적으로 차단됩니다. 현재 및 이전 릴리스에서 개발자 모드 패키지를 차단 해제하려면 WSUS 에서 다음 KB 를 허용해야 합니다: 4016509, 3180030, 3197985
3. 설정 > 업데이트 및 보안 > Windows Update 에서 Windows 업데이트를 확인합니다.
4. 설정 > 시스템 > 앱 & 기능 > 옵션 기능 관리 > 기능 추가에 Windows Developer Mode 패키지가 있는지 확인합니다. 이 패키지가 없으면 컴퓨터에 맞는 올바른 패키지를 찾을 수 없습니다.

위의 단계를 수행한 후 개발자 모드를 비활성화했다가 다시 활성화하여 수정 사항을 확인합니다.

패키지를 설치하지 못함

"개발자 모드 패키지 설치에 실패했습니다. 에러 코드 0x80004005 자세히 보기"

이 오류는 Windows 빌드와 개발자 모드 패키지 간의 비호환성으로 인해 발생할 수 있습니다.

해결 방법:

1. 설정 > 업데이트 및 보안 > Windows Update 에서 Windows 업데이트를 확인합니다.
2. 컴퓨터를 다시 부팅하여 모든 업데이트가 적용되었는지 확인합니다.

그룹 정책 또는 레지스트리 키를 사용하여 장치 사용

대부분의 개발자의 경우 설정 앱을 사용하여 장치에서 디버깅을 사용하도록 설정하려고 합니다. 자동 테스트와 같은 특정 시나리오에서는 Windows 10 데스크톱 장치를 개발할 수 있도록 다른 방법을 사용할 수 있습니다. 이러한 단계에서는 SSH 서버를 사용하거나 장치를 원격 배포 및 디버깅 대상으로 지정할 수 없습니다.

Windows 10 Home 이 없는 경우 gpedit.msc 를 사용하여 장치를 사용하도록 그룹 정책을 설정할 수 있습니다. Windows 10 Home 이 있는 경우 regedit 또는 PowerShell 명령을 사용하여 장치를 사용하도록 레지스트리 키를 직접 설정해야 합니다.

gpedit 을 이용하여 디바이스 활성화

1. Gpedit.msc 를 실행합니다.
2. 로컬 컴퓨터 정책 > 컴퓨터 구성 > 관리 템플릿 > Windows 구성 요소 > 앱 패키지 배포
3. 사이드 로딩을 활성화하려면 다음과 같이 정책을 수정합니다.

- 신뢰할 수 있는 모든 앱 설치 허용

  또는 개발자 모드를 활성화하려면 다음과 같이 정책을 수정합니다.

- 신뢰할 수 있는 모든 앱 설치 허용
- 통합 개발 환경(IDE)에서 UWP 앱을 개발하고 설치 허용

4. 장치를 재부팅합니다.

Regedit 을 이용하여 장치 활성화

1. regedit 을 실행합니다.

2. 사이드로딩을 활성화하려면 다음 DWORD 의 값을 1 로 설정합니다.
   HKLM₩SOFTWARE₩Microsoft₩Windows₩CurrentVersion₩AppModelUnlock₩AllowAllTruestedApps
   또는 개발자 모드를 활성화하려면 다음 DWORD 의 값을 1 로 설정합니다.

   HKLM₩SOFTWARE₩Microsoft₩Windows₩CurrentVersion₩AppModelUnlock₩AllowDevelopmentWithoutDevLicense

PowerShell 을 이용하여 장치 활성화

1. 개발자 권한으로 PowrShell 을 실행합니다.
2. 사이드로딩을 활성화하려면 다음 명령을 입력합니다.

```
PS C:₩WINDOWS₩system32> reg add
"HKEY_LOCAL_MACHINE₩SOFTWARE₩Microsoft₩Windows₩CurrentVersion₩AppModelUnlock" /t REG_DWORD /f /v
"AllowAllTrustedApps" /d "1"
```

또는 개발자 모드를 활성화하려면 다음 명령을 입력합니다.

```
PS C:₩WINDOWS₩system32> reg add
"HKEY_LOCAL_MACHINE₩SOFTWARE₩Microsoft₩Windows₩CurrentVersion₩AppModelUnlock" /t REG_DWORD /f /v "AllowDevelopmentWithoutDevLicense"
/d "1"
```

장치를 Windows 8.1 에서 Windows 10 으로 업그레이드하기

Windows 8.1 장치에서 앱을 만들거나 빼낼 때는 개발자 라이선스를 설치해야 합니다. 장치를 Windows 8.1 에서 Windows 10 으로 업그레이드해도 이 정보는 그대로 유지됩니다. 다음 명령을 실행하여 업그레이드된 Windows 10 장치에서 이 정보를 제거합니다. Windows 8.1 에서 Windows 10, 버전 1511 이상으로 직접 업그레이드하는 경우에는 이 단계가 필요하지 않습니다.

개발자 라이선스 등록 해제

1. 관리자 권한으로 PowerShell 을 실행합니다.
2. 다음 명령을 입력합니다.

Unregister-windowsdeveloperlicense

이 작업을 마친 후에는 이 항목에 설명된 대로 장치에서 계속 개발할 수 있도록 설정해야 합니다. 이렇게 하지 않으면 앱을 디버깅하거나 해당 앱에 대한 패키지를 만들 때 다음 오류를 비롯한 여러 오류가 발생할 수 있습니다.

에러 : DEP0700 : 앱 등록에 실패했습니다.

개발자 계정 생성

2/18/2021

지금 개발자 계정에 등록하면 Microsoft Store 로 앱을 가져오고 다른 Microsoft 프로그램에 참여할 수 있습니다.

지금 가입하세요!

개발자 계정 개설

Microsoft 는 개인용 계정과 회사용 계정을 제공합니다. 가입 과정의 개요를 확인하고 어떻게 되는 것인지 확인하세요.

앱 이름 넣기

개발자 계정을 개설하면 바로 이름을 등록하여 앱을 만들 수 있습니다.

Windows 앱을 만들기 위한 Visual Studio 템플릿

2/18/2021

Visual Studio 2019 는 C#이나 C++로 앱을 만들 때 도움이 되는 다양한 프로젝트 및 아이템 템플릿을 제공합니다. 이번 자료는 그러한 템플릿들에 대하여 설명하고 당신의 시나리오에 맞게 선택을 도와줄 수 있습니다.

- 프로젝트 템플릿에는 프로젝트 파일, 코드 파일 및 앱 또는 앱에서 로드하여 사용할 수 있는 구성 요소를 빌드하도록 구성된 기타 자산이 포함됩니다.
- 아이템 템플릿은 일반적으로 사용되는 코드와 개발 시간을 줄이기 위해 프로젝트에 추가할 수 있는 XAML 을 포함하는 프로젝트 파일입니다. 예를 들어 항목 템플릿을 사용하여 새 창, 페이지 또는 컨트롤을 앱에 추가할 수 있습니다

WinUI 템플릿

Windows UI 라이브러리 (WinUI)는 데스크톱(.NET, 네이티브 Win32) 이나 UWP 플랫폼 등의 Windows 앱을 위한 최신식 네이티브 유저 인터페이스 플랫폼입니다. WinUI 3(현재 개발자 프리뷰)는 WinUI 의 최근 주 버전이며, WinUI 를 Windows 앱을 위한, 완전한 UX 프레임워크로 변환시킵니다.

WinUI 3 에는 WinUI 기반 인터페이스를 가지는 앱을 위한 프로젝트/아이템 템플릿을 제공하는 Visual Studio 2019 용 VSIX 패키지가 포함되어 있습니다. WinUI 3 VSIX 패키지와 제공되는 프로젝트 템플릿에 대하여 더 알아보시려면 WinUI 3 Preview 4 설치를 보십시오.

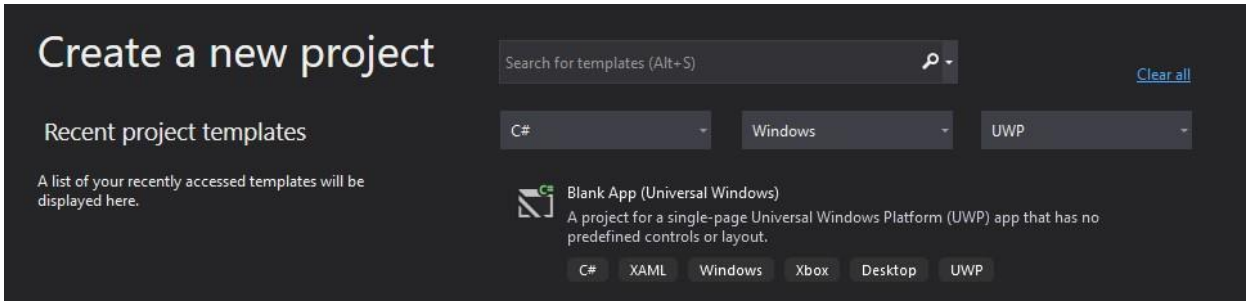
> **IMPORTANT**
>
> 관련 Visual Studio 템플릿을 포함한 WinUI 3 는 현재 개발자 커뮤니티의 피드백을 수집하고 초기 평가를 위한 개발자 미리보기로 제공됩니다. 지금은 생산성 앱에 사용해서는 안 됩니다.

UWP 템플릿

Visual Studio 는 C# 또는 C++로 UWP 앱을 만들기 위한 다양한 프로젝트 템플릿을 제공합니다. 이러한 프로젝트 템플릿을 사용하려면 Visual Studio 를 설치할 때 범용 Windows 플랫폼 개발 워크로드를 포함해야 합니다. C++ 프로젝트 템플릿의 경우 범용 Windows 플랫폼 개발 워크로드에 대한 C++(v142) 범용 Windows 플랫폼 도구(옵션)도 포함해야 합니다.

C# 및 UWP 프로젝트 템플릿

UWP C# 프로젝트 템플릿에 접근하려면 새 프로젝트에서 언어를 C#, 플랫폼을 Windows, 프로젝트 타입을 UWP 로 필터를 설정합니다.



C# UWP 앱을 위하여 다음 프로젝트템플릿들을 사용할 수 있습니다.

템플릿	설명
빈 앱 (범용 Windows)	UWP 앱을 생성합니다. 생성된 프로젝트는 Windows.UI.Xaml.Controls.Page 클래스에서 파생된 기본 페이지를 가집니다. 그것으로 UI 를 만들기 시작할 수 있습니다.
단위 테스트 앱 (범용 Windows)	UWP 앱을 위한 C# 단위 테스트 프로젝트를 생성합니다. 추가적인 정보는 단위 테스트 C# 코드를 참조하세요.

C# UWP 앱의 구성 요소를 만들기 위해 다음 프로젝트 템플릿들을 사용할 수 있습니다.

템플릿	설명
클래스 라이브러리 (범용 Windows)	관리되는 코드로 작성된 다른 UWP 응용프로그램에서 사용할 수 있는 관리 클래스 라이브러리(DLL)를 C#에 만듭니다.
Windows 런타임 구성 요소 (범용 Windows)	앱이 작성된 프로그래밍 언어에 관계없이 모든 UWP 앱에서 사용할 수 있는 Windows Runtime 구성 요소를 C#에 생성합니다.
선택적 코드 패키지 (범용 Windows)	앱에서 로드할 수 있는 실행 파일 C# 코드가 있는 선택적 패키지를 생성합니다. 자세한 내용은 실행 코드가 있는 선택적 패키지를 참조하십시오.

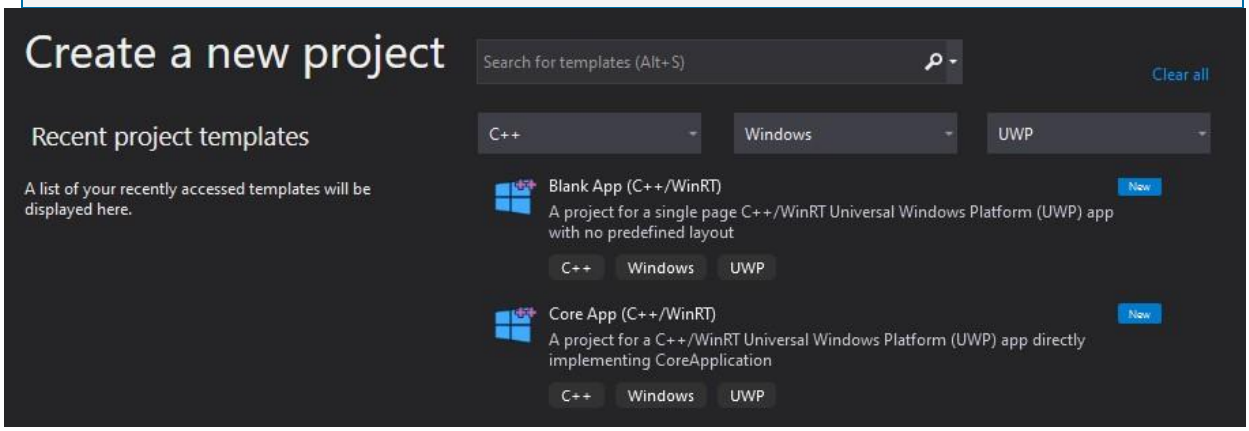
C++ 및 UWP 프로젝트 템플릿

C++ UWP 앱을 만들기 위해서 사용할 수 있는 2 가지 다른 기술이 있습니다.

- 권장되는 기술은 C++/WinRT 입니다. 이것은 전적으로 헤더 파일에 구현된 C++ 언어 프로젝션으로, 최신 WinRT API 에 대한 퍼스트 클래스 액세스를 제공하도록 설계되었습니다.
- 더 이전의 기술인 C++/CX 확장 집합을 사용할 수도 있습니다. C++/CX 는 지금도 지원되지만, C++/WinRT 를 더 권장합니다.

UWP C++ 프로젝트 템플릿에 접근하려면 새 프로젝트에서 언어를 C++, 플랫폼을 Windows, 프로젝트 타입을 UWP 로 필터를 설정합니다.

> **NOTE**
> 기본적으로 Visual Studio 에서 UWP 개발 워크로드는 C++/CX 에만 액세스할 수 있습니다. C++/WinRT 프로젝트 템플릿을 사용하려면 C++/WinRT VSIX 패키지를 설치해야 합니다.



C++ UWP 앱을 만들기 위해 다음 프로젝트 템플릿들을 사용할 수 있습니다.

템플릿	설명
빈 앱 (C++/WinRT)	XAML UI 를 가지는 C++/WinRT UWP 앱을 생성합니다. 생성된 프로젝트는 Windows.UI.Xaml.Controls.Page 클래스에서 파생된 기본 페이지를 가집니다. 그것으로 UI 를 만들기 시작할 수 있습니다.
코어 앱 (C++/WinRT)	CoreApplication 을 사용하여 XAML 사용자 인터페이스 대신 다양한 UI 프레임워크와 통합하는 C++/Win UWP 앱을 생성합니다. 이 프로젝트 템플릿을 사용하여 DirectX 를 사용하는 간단한 게임을 만드는 방법을 보여주는 자세한 내용은 DirectX 를 사용하여 간단한 UWP 게임 만들기를 참조하십시오.
빈 앱 (범용 Windows - C++/CX)	XAML UI 를 가지는 C++/WinRT UWP 앱을 생성합니다. 생성된 프로젝트는 Windows.UI.Xaml.Controls.Page 클래스에서 파생된 기본 페이지를 가집니다. 그것으로 UI 를 만들기 시작할 수 있습니다. (주: 원본 자료에 이렇게 써 있었음)
DirectX 11 and XAML 앱 (범용 Windows - C++/CX)	DirectX 11 과 SwapCahinPanel 을 사용하여 XAML UI 컨트롤을 사용할 수 있는 UWP 앱을 생성합니다. 추가적인 정보는 DirectX 게임 프로젝트 템플릿을 참조하십시오.
DirectX 11 앱 (범용 Windows - C++/CX)	DirectX 11 을 사용하는 UWP 앱을 생성합니다. 추가적인 정보는 DirectX 게임 프로젝트 템플릿을 참조하십시오.
DirectX 12 앱 (범용 Windows - C++/CX)	DirectX 12 를 사용하는 UWP 앱을 생성합니다. 추가적인 정보는 DirectX 게임 프로젝트 템플릿을 참조하십시오.
단위 테스트 앱 (범용 Windows - C++/CX)	C++/CX UWP 앱 단위 테스트 프로젝트를 만듭니다. 추가적인 정보는 C++ DLL 테스트하는 방법을 참조하십시오.

C++ UWP 앱의 구성 요소를 만들기 위해 다음 프로젝트 템플릿들을 사용할 수 있습니다.

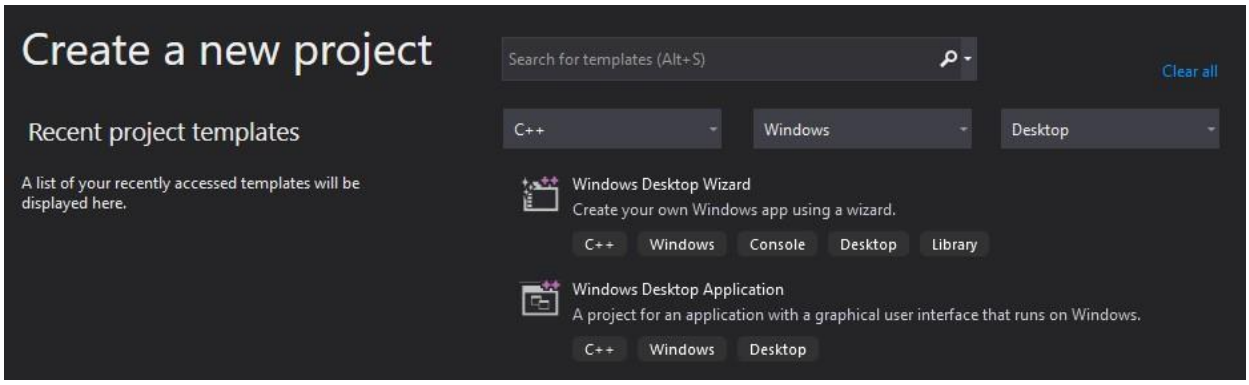
템플릿	설명
Windows 런타임 구성 요소 (C++/WinRT)	사용 언어에 관계없이 UWP 에서 사용할 수 있는 C++/WinRT Windows 런타임 구성 요소를 만듭니다.
Windows 런타임 구성 요소 (범용 Windows)	사용 언어에 관계없이 UWP 에서 사용할 수 있는 C++/CX Windows 런타임 구성 요소를 만듭니다.
DLL (범용 Windows)	UWP 에서 사용할 수 있는 C++/CX 동적 링킹 라이브러리를 만드는 프로젝트입니다. 추가적인 정보는 DLLs(C++/CX)를 참조하십시오.
정적 라이브러리 (범용 Windows)	UWP 에서 사용할 수 있는 C++/CX 정적 라이브러리(LIB)를 만드는 프로젝트입니다. 추가적인 정보는 정적 라이브러리(C++/CX)를 참조하십시오.

C++/Win32 템플릿

Visual Studio 는 기본 C++ 및 Win32 API 에 직접 액세스할 수 있는 데스크톱 Windows 앱을 만들기 위한 다양한 프로젝트 템플릿을 제공합니다. 이러한 프로젝트 템플릿을 사용하려면 Visual Studio 를 설치할 때 C++ 워크로드가 포함된 데스크톱 개발을 포함해야 합니다. 이 워크로드에는 데스크톱 애플리케이션, 콘솔 애플리케이션 및 라이브러리를 구축하기 위한 프로젝트 템플릿이 포함됩니다.

데스크톱 앱을 위한 프로젝트 템플릿

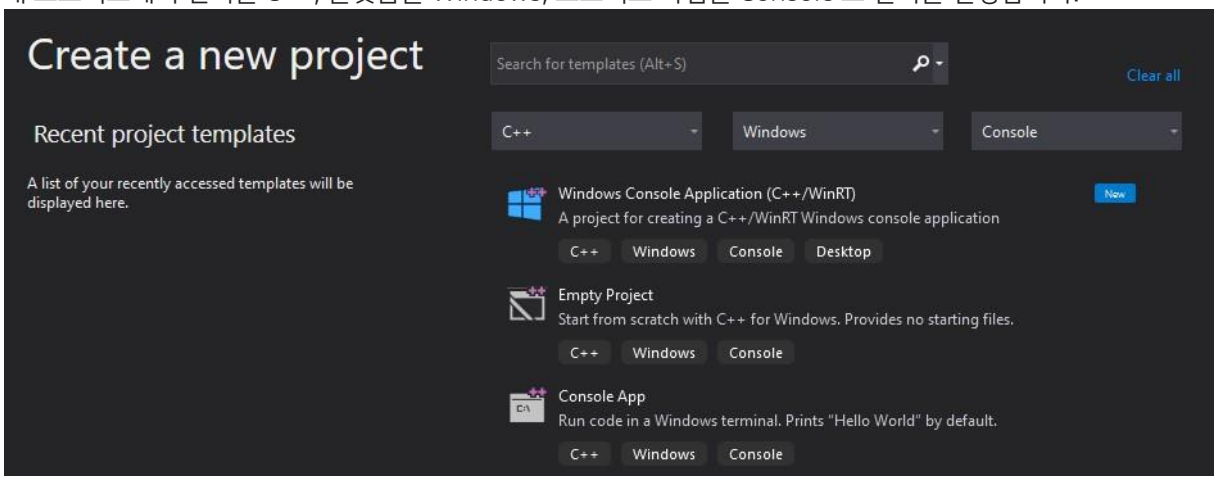
새 프로젝트에서 언어를 C++, 플랫폼을 Windows, 프로젝트 타입을 데스크톱으로 필터를 설정합니다.



템플릿	설명
Windows 데스크톱 앱	기본 Windows 데스크톱 앱을 C++로 만듭니다. 추가적인 정보는 가이드: 전통적 Windows 데스크톱 앱 만들기를 참고하십시오.
Windows 데스크톱 마법사	클래식 Windows 데스크톱 앱, 콘솔 앱, 동적 링크 라이브러리(DLL) 또는 정적 라이브러리 중 하나를 만드는 데 사용할 수 있는 단계별 마법사를 제공합니다. 자세한 내용은 Windows 데스크톱 마법사 및 가이드: 전통적 Windows 데스크톱 응용 프로그램 만들기를 참조하십시오.
Windows 앱 패키징 프로젝트	MSIX 패키지로 데스크톱 앱을 빌드하는 데 사용할 수 있는 프로젝트를 만듭니다. 이를 통해 최신 배포 환경, 패키지 확장을 통해 Windows 10 기능과 통합할 수 있는 기능 등이 제공됩니다. 자세한 내용은 Windows 응용 프로그램 패키징 프로젝트를 참조하십시오.

콘솔 앱 프로젝트 템플릿

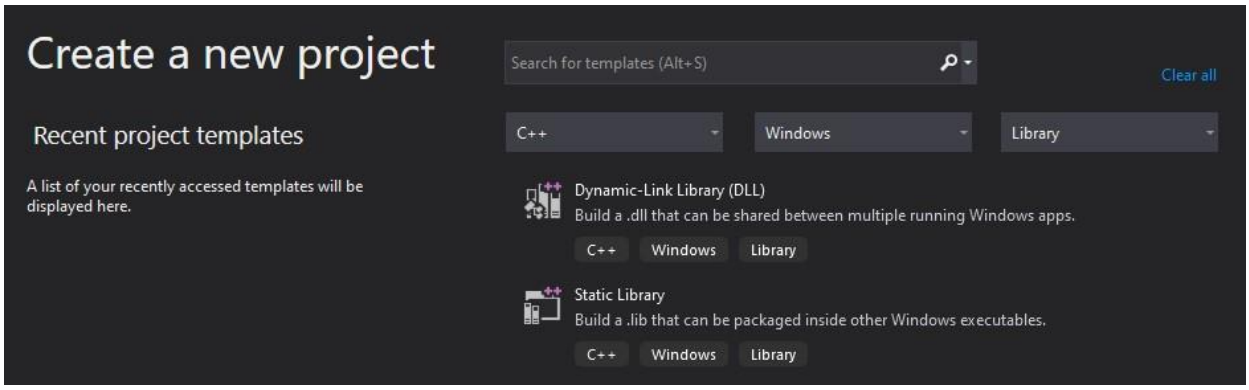
새 프로젝트에서 언어를 C++, 플랫폼을 Windows, 프로젝트 타입을 Console 로 필터를 설정합니다.



템플릿	설명
Windows 콘솔 앱 (C++/WinRT)	UI 없는 C++/WinRT 콘솔 앱을 생성합니다. 추가적인 정보는 C++/WinRT 빠른 시작을 참조하십시오. 이 프로젝트 템플릿은 C++/WinRT VSIX 를 필요로 합니다.
콘솔 앱	UI 없는 콘솔 앱을 생성합니다. 추가적인 정보는 가이드: 표준 C++ 프로그램 만들기를 참조하십시오.
빈 프로젝트	앱, 라이브러리, DLL 을 만들기 위한 빈 프로젝트입니다. 필요한 코드나 리소스를 직접 추가해야 합니다.

라이브러리를 위한 프로젝트 템플릿

새 프로젝트에서 언어를 C++, 플랫폼을 Windows, 프로젝트 타입을 라이브러리로 필터를 설정합니다.



템플릿	DESCRIPT ION
동적 링킹 라이브러리 (DLL)	동적 링킹 라이브러리를 만드는 프로젝트입니다. 추가적인 정보는 가이드: DLL 만들고 사용하기를 참조하십시오.
정적 라이브러리	정적 라이브러리(LIB)를 만드는 프로젝트입니다. 추가적인 정보는 가이드: 라이브러리 만들고 사용하기를 참조하십시오.

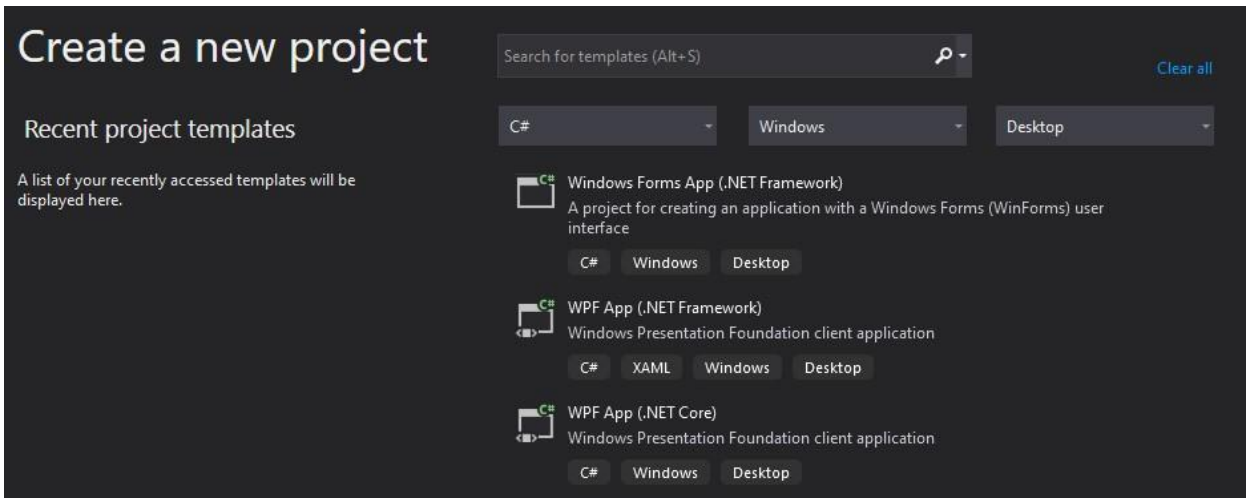
네이티브 C++와 Win32 를 위한 아이템 템플릿

C++ 프로젝트 템플릿은 새 파일 및 리소스 추가와 같은 작업을 할 수 있도록 많은 아이템 템플릿을 포함합니다. 전체 목록은 Visual C++ 사용 및 새 아이템 템플릿 추가를 참조하십시오.

.NET 템플릿

Visual Studio 는 .NET 및 C#을 사용하는 데스크톱 Windows 앱을 빌드하기 위한 다양한 프로젝트 템플릿을 제공합니다. 이러한 프로젝트 템플릿을 사용하려면 Visual Studio 를 설치할 때 .NET 데스크톱 개발 워크로드를 포함해야 합니다.

새 프로젝트에서 언어 C#, 플랫폼 Windows, 프로젝트 타입 데스크톱으로 필터를 설정합니다.



C#과 .NET 을 사용하는 앱을 만들 때 다음 프로젝트 템플릿을 이용할 수 있습니다.

템플릿	설명
WPF 앱 (.NET Core)	.NET Core 를 대상으로 하는 WPF 앱을 만듭니다. WPF 앱 만들기에서 사용법이 안내됩니다.
WPF 앱 (.NET Framework)	.NET Framework 를 대상으로 하는 WPF 앱을 만듭니다. 튜토리얼: 첫 WPF 앱 만들기에서 사용법이 안내됩니다.
Windows Forms 앱 (.NET Core)	.NET Core 대상의 Windows Forms 앱을 만듭니다.
Windows Forms 앱 (.NET Framework)	.NET Framework 대상의 Windows Forms 앱을 만듭니다. Visual Studio 에서 C#으로 Windows Forms 앱 만들기에서 사용법이 안내됩니다.
Windows 애플리케이션 패키징 프로젝트	WPF 또는 Windows Forms 앱을 MSIX 패키지로 빌드하는 데 사용할 수 있는 프로젝트를 만듭니다. 이를 통해 최신 배포 환경, 패키지 확장을 통해 Windows 10 기능과 통합할 수 있는 기능 등이 제공됩니다. 자세한 내용은 Windows 응용 프로그램 패키징 프로젝트를 참조하십시오.

데스크톱 앱 현대화하기

2/18/2021

Windows 10 및 UWP(Universal Windows Platform)는 데스크톱 앱에서 최신 환경을 제공하는 데 사용할 수 있는 다양한 기능을 제공합니다. 이러한 기능 중 대부분은 다른 플랫폼에 맞게 애플리케이션을 다시 작성할 필요 없이 자신만의 속도로 데스크톱 애플리케이션에 적용할 수 있는 모듈형 구성 요소로 제공됩니다. Windows 10 및 UWP 에서 적용할 부분을 선택하여 기존 데스크톱 앱을 향상시킬 수 있습니다.

이 자료에서는 현재 데스크톱 앱에서 사용할 수 있는 Windows 10 및 UWP 기능에 대해 설명합니다. 이 문서에 설명된 많은 기능을 사용하기 위해 기존 앱을 현대화하는 방법을 설명하는 자습서는 여기를 참조하십시오.

> **NOTE**
> 데스크톱 앱을 Windows 10 으로 마이그레이션하는 데 도움이 필요하십니까? Desktop App Assure 서비스는 Windows 10 에 애플리케이션을 포팅하는 개발자에게 비용 없이 직접 지원을 제공합니다. 이 프로그램은 모든 ISV 및 적격 기업에서 사용할 수 있습니다. 자격 및 프로그램 자체에 대한 자세한 내용은 여기를 참조하십시오. 지금 시작하려면 요청을 제출하세요.

Windows UI 라이브러리

Windows UI 라이브러리는 Windows 10 앱에 대한 컨트롤 및 기타 사용자 인터페이스 요소를 제공하는 NuGet 패키지 집합입니다. WinUI 는 Windows 10 의 하위 버전을 대상으로 하는 UWP 앱을 위해 WinRT XAML 컨트롤의 새로운 버전과 업데이트된 버전을 제공하는 툴킷으로 시작했습니다. WinUI 의 범위가 확장되어 이제는 UWP, .NET 및 기본 Win32 에서 Windows 10 앱을 위한 최신 기본 사용자 인터페이스(UI) 플랫폼이 되었습니다.

다음과 같은 방법으로 WinUI 를 데스크톱 앱에 사용할 수 있습니다.

- XAML Islands 를 이용하여 기존 WPF, Windows Forms, C++/Win32 앱을 업데이트하고 WinUI 2.x 컨트롤로 호스트합니다.
- WinUi 3.0 (현재 프리뷰)를 사용하여 완전히 WinUI 기반 UI 를 가지며 관리된 C#/.NET Core and native C++/Win32 데스크톱 앱을 제작합니다.

Windows UI (WinUI) 라이브러리

MSIX 패키지

MSIX 는 UWP, WPF, Windows Forms, Win32 앱을 포함한 모든 Windows 앱에 범용 패키지 환경을 제공하는 최신 Windows 앱 패키지 포맷입니다. MSIX 는 MSI, .appx, App-V 및 ClickOnce 설치 기술의 최고 측면을 통합하여 현대적이고 안정적인 패키징 환경을 제공합니다.

데스크톱 Windows 애플리케이션을 MSIX 패키지로 패키징하면 강력한 설치 및 업데이트 환경, 유연한 기능 시스템을 갖춘 관리 보안 모델, 마이크로소프트 Store 지원, 엔터프라이즈 관리 및 다양한 사용자 지정 배포 모델에 액세스할 수 있습니다.

더 많은 정보는 MSIX 문서의 데스크톱 앱 패키지로 만들기를 참고하십시오.

.NET Core 3

.NET Core 3 는 .NET Core 의 최신 주 릴리스입니다. 이번 릴리스의 주 업데이트는 Windows Forms 및 WPF 앱을 포함한 Windows 데스크톱 앱 지원입니다. .NET Core 3 에서 새로운 Windows 데스크톱 앱과 기존 Windows 데스크톱 앱을 실행하고 .NET Core 가 제공하는 모든 혜택을 누릴 수 있습니다. XAML Islands 에서 호스팅되는 WinRT XAML 컨트롤은 .NET Core 3 를 대상으로 하는 Windows Forms 및 WPF 앱에서도 사용할 수 있습니다.

더 많은 정보는 .NET Core 3.0 의 새로운 기능을 참고하십시오.

Windows 런타임 API

WPF, Windows Forms 또는 C++ Win32 데스크톱 앱에서 직접 많은 Windows Runtime API 를 호출하여 Windows 10 사용자를 위한 최신 환경을 통합할 수 있습니다. 예를 들어 Windows Runtime API 를 호출하여 데스크톱 앱에 토스트 알림을 추가할 수 있습니다.

더 많은 정보는 데스크톱 앱에서 Windows 런타임 API 호출을 참고하십시오.

WinRT XAML 컨트롤 (XAML Islands) 호스팅

Windows 10 버전 1903 부터는 WPF, Windows Forms 또는 HWND(윈도우 핸들)와 연결된 C++ Win32 앱의 UI 요소에 UWP XAML 컨트롤을 직접 추가할 수 있습니다. 따라서 Fluent Design System 을 지원하는 Windows Ink 및 컨트롤과 같은 최신 UWP 기능을 데스크톱 앱의 창 및 기타 디스플레이 표면에 완전히 통합할 수 있습니다. 이 개발자의 시나리오를 XAML Islands 라고 부르기도 합니다.

더 많은 정보는 데스크톱 앱에서 WinRT XAML 컨트롤을 참고하십시오.

데스크톱 앱에서 Visual layer 사용

이제 UWP 가 아닌 데스크톱 앱에서 Windows Runtime API 를 사용하여 WPF, Windows Forms 및 C++ Win32 앱의 모양, 느낌 및 기능을 향상하고 UWP 를 통해서만 사용할 수 있는 최신 Windows 10 UI 기능을 활용할 수 있습니다. 이 기능은 XAML Islands 를 사용하여 호스팅할 수 있는 기본 제공 WinRT XAML 컨트롤을 벗어나는 사용자 지정 환경을 생성해야 할 때 유용합니다.

더 많은 정보는 Visual layer 를 사용하여 데스크톱 앱 현대화하기를 참고하십시오.

패키지 ID 를 가진 앱에 사용할 수 있는 다른 기능들

일부 최신 Windows 10 환경은 패키지 ID 가 있는 데스크톱 앱에서만 사용할 수 있습니다. 이러한 기능에는 특정 Windows Runtime API, 패키지 확장 및 UWP 구성 요소가 포함됩니다. 자세한 내용은 패키지 ID 가 필요한 기능을 참조하십시오.

데스크톱 앱에서 ID 를 부여하는 방법은 다음과 같습니다.

- MSIX 패키지로 만들기. MSIX 는 모든 Windows 앱, WPF, Windows Forms 및 Win32 앱에 범용 패키지 환경을 제공하는 최신 앱 패키지 포맷입니다. 강력한 설치 및 업데이트 환경, 유연한 기능 시스템을 갖춘 관리형 보안 모델, Microsoft Store 지원, 엔터프라이즈 관리 및 다양한 맞춤형 배포 모델을 제공합니다. 자세한 내용은 MSIX 설명서의 데스크톱 앱을 패키지로 만들기를 참조하십시오.
- Windows 10(윈도우 10) 버전 2004 부터 데스크톱 앱 배포에 MSIX 패키지를 적용할 수 없는 경우 패키지 매니페스트만 포함된 sparse MSIX 패키지를 생성하여 패키지 ID 를 부여할 수 있습니다. 자세한 내용은 비패키지 데스크톱 앱에 ID 부여를 참조하십시오.

데스크톱 앱에 최적화된 WinRT XAML 컨트롤

데스크톱 장치 제품군만을 대상으로 하는 UWP 앱을 구축하거나 WPF, Windows Forms 또는 C++ Win32 데스크톱 앱에서 WinRT XAML 컨트롤을 사용하고자 하는 경우, 다음과 같이 새롭게 업데이트된 WinRT XAML 컨트롤은 Fluent Design System 을 통해 데스크톱에 최적화된 환경을 제공하도록 설계되었습니다. 이러한 컨트롤은 Windows 10, 버전 1809(2018.10 업데이트, v10.0.17763)에 도입되었습니다.

컨트롤	설명
MenuBar	CommandBar 에서 허용하는 것보다 더 많은 구성 또는 그룹이 필요할 수 있는 앱에 대한 명령 집합을 쉽고 빠르게 노출할 수 있는 방법을 제공합니다.
DropDownButton	더 많은 옵션을 포함하는 플라이아웃이 첨부되어 있음을 시각적 지표로 v 모양을 표시합니다.

SplitButton	별도로 호출할 수 있는 두 개의 부분으로 된 버튼을 제공합니다. 하나는 표준 버튼처럼 작동하여 즉시 작업을 실행합니다. 다른 하나는 사용자가 선택할 수 있는 추가 옵션이 포함된 플라이아웃을 호출합니다.
ToggleSplitButton	별도로 호출할 수 있는 두 개의 부분으로 된 버튼을 제공합니다. 하나는 ON/OFF 버튼과 같습니다. 다른 하나는 사용자가 선택할 수 있는 추가 옵션이 포함된 플라이아웃을 호출합니다.
CommandBarFlyout	UI 캔버스에 있는 항목의 컨텍스트에서 일반 사용자 작업을 표시할 수 있습니다.
ComboBox	이제 사용자가 컨트롤에 나열되지 않은 값을 입력할 수 있도록 콤보 박스를 편집할 수 있습니다.
TreeView	이제 데이터 바인딩, 항목 템플릿 및 드래그/드롭을 사용하도록 트리 보기를 구성할 수 있습니다.
DataGridView	데이터 컬렉션을 행과 열에 유연하게 표시할 수 있는 방법을 제공합니다. 이 컨트롤은 Windows 커뮤니티 툴킷에서 사용할 수 있습니다.

현대 데스크톱 앱의 다른 기술들

Microsoft 그래프

Microsoft Graph 는 수백만 사용자의 데이터와 상호작용하는 조직 및 소비자를 위한 앱을 구축하는 데 사용할 수 있는 API 모음입니다. Microsoft Graph 는 REST API 및 클라이언트 라이브러리를 표시하여 다음 데이터에 액세스합니다.

- Azure Active Directory
- Microsoft 365 오피스 앱: SharePoint, OneDrive, Outlook/Exchange, Microsoft Teams, OneNote, Planner, and Excel
- 기업 이동/보안 서비스: Identity Manager, Intune, Advanced Threat Analytics, Advanced Threat Protection
- Windows 10 서비스: 활동, 장치

더 많은 정보는 Microsoft 그래프 문서를 참조해 주세요.

적응형 카드

Adaptive Cards 는 여러 장치와 플랫폼에서 공통적이고 일관된 방식으로 카드 기반 UI 콘텐츠를 교환하는 데 사용할 수 있는 개방형 교차 플랫폼 프레임워크입니다.

더 많은 정보는 적응형 카드 문서를 참조해 주세요.

데스크톱 앱에서 Windows 런타임 API 호출

2/18/2021

UWP(Universal Windows Platform) API 를 사용하여 Windows 10 사용자를 위해 켜지는 데스크톱 앱에 최신 환경을 추가할 수 있습니다.

먼저 필요한 참조로 프로젝트를 설정합니다. 그런 다음 코드에서 Windows Runtime API 를 호출하여 Windows 10 환경을 데스크톱 앱에 추가합니다. Windows 10 사용자에 대해 별도로 빌드하거나 실행 중인 Windows 버전에 관계없이 동일한 이진 파일을 모든 사용자에게 배포할 수 있습니다.

일부 Windows Runtime API 는 패키지 ID 가 있는 데스크톱 앱에서만 지원됩니다. 자세한 내용은 사용 가능한 Windows 런타임 API 를 참조하십시오.

.NET 프로젝트에서 Windows 런타임 API 를 사용하도록 수정

There are several options for .NET projects:

- .NET 5 Preview 8 부터는 프로젝트 파일에 TPM(Target Framework Moniker)을 추가하여 WinRT API 에 액세스할 수 있습니다. 이 옵션은 Windows 10 버전 1809 이상을 대상으로 하는 프로젝트에서 지원됩니다.
- 이전 버전의 .NET 의 경우 NuGet 패키지 **Microsoft.Windows.SDK.Contracts** 를 설치하여 프로젝트에 필요한 모든 참조를 추가할 수 있습니다. 이 옵션은 Windows 10 버전 1803 이상을 대상으로 하는 프로젝트에서 지원됩니다.
- .NET 5 Preview 8 와 .NET 의 이전 버전을 모두 대상으로 하는 경우, 두 옵션 모두 사용하도록 프로젝트 파일을 구성할 수 있습니다.

.NET 5: Target Framework Moniker 옵션 사용

이 옵션은 .NET 5(또는 이후 버전 릴리스) 및 Windows 10 1809 이상 버전 OS 릴리스 대상 프로젝트에서만 지원됩니다. 이 시나리오에 대한 자세한 배경 정보는 이 블로그 게시물을 참조하십시오.

1. Visual Studio 에서 프로젝트를 열고 솔루션 탐색기의 프로젝트를 우클릭하여 프로젝트 파일 수정을 선택하면, 다음과 같이 생긴 것을 볼 수 있습니다.

```
<Project Sdk="Microsoft.NET.Sdk.WindowsDesktop">
  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net5.0</TargetFramework>
    <UseWindowsForms>true</UseWindowsForms>
  </PropertyGroup>
</Project>
```

2. <TargetFramework> 다음의 값을 다음 중 하나로 수정합니다.

   net5.0-windows10.0.17763.0 (Windows 10, version 1809 대상)
   net5.0-windows10.0.18362.0 (Windows 10, version 1903 대상)
   net5.0-windows10.0.19041.0 (Windows 10, version 2004 대상)
   이를 테면 Windows 10 버전 2004 를 대상으로 하는 경우 다음과 같이 수정합니다.

```
<TargetFramework>net5.0-windows10.0.19041.0</TargetFramework>
```

3. 저장하고 프로젝트 파일을 닫습니다.

이전 버전 .NET: Microsoft.Windows.SDK.Contracts NuGet 패키지 설치

앱이 .NET. Core 3.x, .NET 5 Preview 7(이하), .NET Framework 를 사용하는 경우 이 선택지를 따릅니다. 타겟이 Windows 10 버전 1803 이후인 경우 지원됩니다.

1. 패키지 레퍼런스가 활성화되어 있어야 합니다.
   a. Visual Studio 에서 도구 -> NuGet 패키지 관리자 -> 패키지 관리자 설정
   b. **PackageReference** 가 기본 패키지 관리 포맷으로 선택되어 있어야 합니다.
2. Visual Studio 에서 프로젝트를 열고 솔루션 탐색기에서 프로젝트 우클릭 후 NuGet 패키지 관리를 선택합니다.
3. NuGet 패키지 관리자 창에서 찾아보기 탭을 선택하고 Microsoft.Windows.SDK.Contracts 를 찾습니다.
4. 페이지를 찾았으면 NuGet 패키지 관리자 창의 오른쪽 창에서 대상 Windows 10 버전을 기준으로 설치할 패키지 버전을 선택합니다.

   10.0.19041.xxxx: (Windows 10, version 2004 대상)
   10.0.18362.xxxx: (Windows 10, version 1903 대상)
   10.0.17763.xxxx: (Windows 10, version 1809 대상)
   10.0.17134.xxxx: (Windows 10, version 1803 대상)
5. 설치를 클릭합니다.

여러 버전의 .NET 대상일 때 구성하기

.NET 5 Preview 8 과 그 이전 버전(.NET Core 3.x, .NET Framework)을 같이 대상으로 하는 경우 Target Framework Moniker 를 사용하여 .NET 5 에 대한 WinRT API 참조를 자동으로 가져오도록 프로젝트 파일을 구성하고 이전 버전에 **Microsoft.Windows.SDK.Contracts** NuGet 패키지를 사용할 수 있습니다.

1. Visual Studio 에서 프로젝트를 열고 솔루션 탐색기의 프로젝트를 우클릭하여 프로젝트 파일 수정을 선택합니다. .NET Core 3.1 을 사용하는 앱은 다음과 같이 보여집니다.

```
<Project Sdk="Microsoft.NET.Sdk.WindowsDesktop">
 <PropertyGroup>
  <OutputType>WinExe</OutputType>
  <TargetFramework>netcoreapp3.1</TargetFramework>
  <UseWindowsForms>true</UseWindowsForms>
 </PropertyGroup>
</Project>
```

2. <TargetFramework> 다음의 값을 수정합니다. 여기서 모든 대상 .NET 버전의 Target Framework Moniker 를 ;으로 구분하여 명시합니다.

- .NET 5 Preview 8 이후:

  net5.0-windows10.0.17763.0 (Windows 10 버전 1809 대상)
  net5.0-windows10.0.18362.0 (Windows 10 버전 1903 대상)
  net5.0-windows10.0.19041.0 (Windows 10 버전 2004 대상)
- .NET Core 3.x: netcoreapp3.0 또는 netcoreapp3.1
- .NET Framework: net46.

  예를 들어 .NET Core 3.1 과 .NET 5 Preview 8 대상은 다음과 같습니다.

```
<TargetFrameworks>netcoreapp3.1;net5.0-windows10.0.19041.0</TargetFrameworks>
```

3. </PropertyGroup> 다음에 어떤 버전의 .NET Core 3.x, .NET Framework 등 타겟을 위해 **Microsoft.Windows.SDK.Contracts** 를 설치하도록 PackageReference 를 추가합니다. 이것은 ItemGroup 의 안에 있어야 합니다. .NET Core3.1 에 대하여 예를 들면 다음과 같습니다.

```
<ItemGroup>
 <PackageReference Condition="'$(TargetFramework)' == 'netcoreapp3.1'"          Include="Microsoft.Windows.SDK.Contracts"
            Version="10.0.19041.0" /> </ItemGroup>
```

전부 수행했다면 프로젝트 파일은 다음과 같이 생겼을 것입니다.

```
<Project Sdk="Microsoft.NET.Sdk.WindowsDesktop">
 <PropertyGroup>
  <OutputType>WinExe</OutputType>
  <TargetFrameworks>netcoreapp3.1;net5.0-windows10.0.19041.0</TargetFrameworks>
  <UseWPF>true</UseWPF>
 </PropertyGroup>
 <ItemGroup>
  <PackageReference Condition="'$(TargetFramework)' == 'netcoreapp3.1'"
            Include="Microsoft.Windows.SDK.Contracts"
            Version="10.0.19041.0" />   </ItemGroup>
</Project>
```

4. 저장하고 프로젝트 파일을 닫습니다.

C++ Win32 프로젝트가 Windows 런타임 API 를 사용하도록 수정

C++/WinRT 를 사용합니다. C++/WinRT 는 윈도우 런타임(WinRT) API 를 위한 완전 표준 현대식 C++17 언어 프로젝션으로, 헤더 파일 기반 라이브러리로 구현되며 최신 윈도우 API 에 대한 최고 수준의 액세스를 제공하도록 설계되었습니다.

C++/WinRT 를 사용하도록 구성하려면:

- 새 프로젝트의 경우, C++/WinRT Visual Studio Extension (VSIX)를 설치하고 거기에 포함된 C++/WinRT 프로젝트 템플릿을 선택합니다.
- 기존 프로젝트의 경우, Microsoft.Windows.CppWinRT NuGet 패키지를 설치합니다.

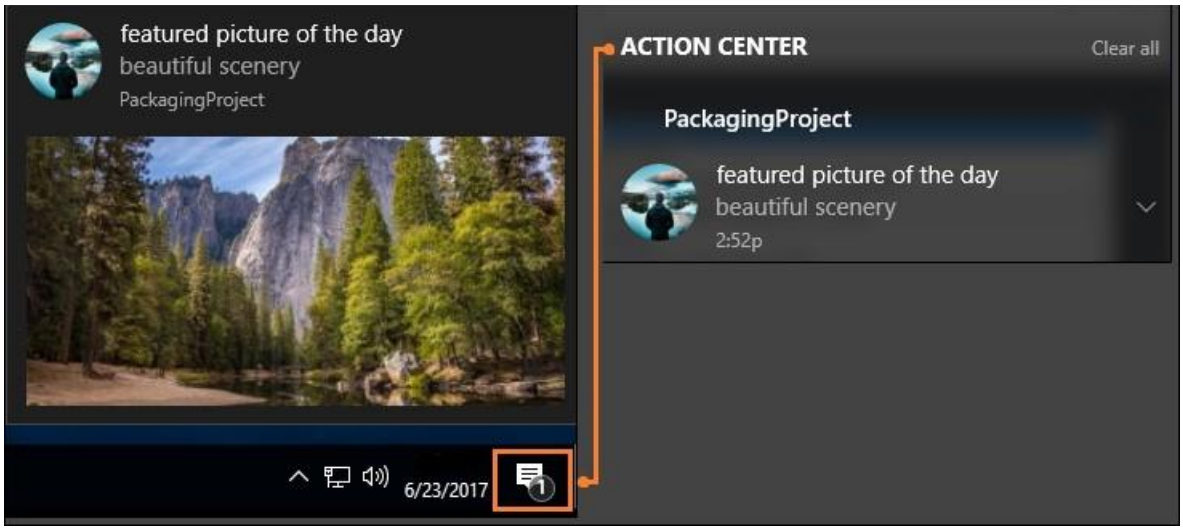
위 선택지에 대하여 더 자세한 사항은 여기를 참조하십시오.

Windows 10 experiences 추가

이제 Windows 10 에서 응용 프로그램을 실행할 때 켜지는 최신 환경을 추가할 준비가 되었습니다. 이 설계 흐름을 사용합니다.

☑ 먼저, 추가하려는 경험(UX)을 결정합니다.

선택할 수 있는 것이 많습니다. 예를 들어 수익화 API 를 사용하여 구매 주문 흐름을 단순화하거나, 다른 사용자가 게시한 새 사진과 같이 공유할 흥미로운 내용이 있을 때 응용 프로그램에 주의를 끌 수 있습니다.



사용자가 메시지를 무시하거나 무시하더라도 알림 센터에서 메시지를 다시 본 다음 메시지를 눌러 응용프로그램을 열 수 있습니다. 이렇게 하면 응용 프로그램에 대한 참여가 증가하고 응용 프로그램이 운영 체제와 긴밀하게 통합된 것처럼 보일 수 있는 추가 비용이 발생합니다. 이러한 경험을 위한 코드는 이 글의 뒷부분에서 보여드리겠습니다.

더 많은 아이디어는 UWP 문서를 보시면 많이 있습니다.

☑ 개선할지 확장할지 결정합니다.

개선 및 확장이라는 용어가 자주 등장하므로 잠시 시간을 내어 각 용어가 정확히 무엇을 의미하는지 설명하겠습니다.

개선은 데스크톱 앱에서 직접 호출할 수 있는 Windows 런타임 API 를 설명하는 데 사용되는 단어입니다(MSIX 패키지를 하도록 설정한 것과 관계없이). Windows 10 환경을 선택했으면 생성에 필요한 API 를 식별한 다음 해당 API 가 이 목록에 표시되는지 확인하십시오. 데스크톱 앱에서 직접 호출할 수 있는 API 목록입니다. API 가 이 목록에 나타나지 않으면 해당 API 와 관련된 기능이 UWP 프로세스 내에서만 실행될 수 있기 때문입니다. UWP 맵 컨트롤이나 Windows Hello 보안 프롬프트와 같은 UWP XAML 을 렌더링하는 API 를 포함하는 경우가 많습니다.

> NOTE
>
> UWP XAML 을 렌더링하는 API 는 일반적으로 데스크톱에서 직접 호출할 수 없지만 다른 방법을 사용할 수도 있습니다. UWP XAML 컨트롤이나 기타 사용자 지정 시각 환경을 호스팅하려면 XAML Islands(Windows 10, 버전 1903 부터 시작) 및 Visual layer(Windows 10, 버전 1803)을 사용할 수 있습니다. 이러한 기능은 패키지 또는 패키지화되지 않은 데스크톱 앱에서 사용할 수 있습니다.

데스크톱 앱을 MSIX 패키지로 패키징하도록 선택한 경우 다른 옵션은 솔루션에 UWP 프로젝트를 추가하여 애플리케이션을 확장하는 것입니다. 데스크톱 프로젝트는 여전히 응용 프로그램의 진입점이지만 UWP 프로젝트는 이 목록에 나타나지 않는 모든 API 에 대한 액세스를 제공합니다. 데스크탑 앱은 앱 서비스를 이용하여 UWP 프로세스와 통신할 수 있으며, 이를 설정하는 방법에 대한 많은 지침이 있습니다. UWP 프로젝트가 필요한 환경을 추가하려면 UWP 구성 요소로 확장을 참조하십시오.

☑ API 계약을 참조합니다.

데스크톱 앱에서 직접 API 를 호출할 수 있는 경우 브라우저를 열고 해당 API 의 참조 항목을 검색합니다. API 요약 아래에는 해당 API 의 API 계약을 설명하는 테이블이 있습니다. 이 표의 예는 다음과 같습니다.

Windows 10 requirements

Device family	Windows 10 (introduced v10.0.10240.0)
API contract	Windows.Foundation.UniversalApiContract (introduced v1)

.NET 기반 데스크톱 앱이 있는 경우 해당 API 계약에 참조를 추가한 다음 해당 파일의 Copy Local 속성을 False 로 설정합니다. C++ 기반 프로젝트가 있는 경우 이 계약이 포함된 폴더의 경로인 추가 포함 디렉터리에 추가하십시오.

☑ 경험을 추가하기 위해 API 를 호출합니다.

앞에서 살펴본 알림 창을 표시할 때 사용하는 코드입니다. 이러한 API 가 이 목록에 표시되므로 이 코드를 데스크톱 앱에 추가하고 지금 바로 실행할 수 있습니다.

```
using Windows.Foundation; using
Windows.System; using
Windows.UI.Notifications; using
Windows.Data.Xml.Dom; ...

private void ShowToast()
{
   string title = "featured picture of the day";    string content = "beautiful scenery";
   string image = "https://picsum.photos/360/180?image=104";    string logo =
"https://picsum.photos/64?image=883";

   string xmlString =    $@"<toast><visual>
     <binding template='ToastGeneric'>
     <text>{title}</text>
     <text>{content}</text>
     <image src='{image}'/>
     <image src='{logo}' placement='appLogoOverride' hint-crop='circle'/>      </binding>
    </visual></toast>";

   XmlDocument toastXml = new XmlDocument();    toastXml.LoadXml(xmlString);

   ToastNotification toast = new ToastNotification(toastXml);

   ToastNotificationManager.CreateToastNotifier().Show(toast); }
```

```
#include <sstream>
#include <winrt/Windows.Data.Xml.Dom.h>
#include <winrt/Windows.UI.Notifications.h>

using namespace winrt::Windows::Foundation; using namespace winrt::Windows::System;
using namespace winrt::Windows::UI::Notifications; using namespace winrt::Windows::Data::Xml::Dom;

void UWP::ShowToast()
{
   std::wstring const title = L"featured picture of the day";    std::wstring const content = L"beautiful scenery";
   std::wstring const image = L"https://picsum.photos/360/180?image=104";    std::wstring const logo = L"https://picsum.photos/64?image=883";

   std::wostringstream xmlString;
   xmlString << L"<toast><visual><binding template='ToastGeneric'>" <<
      L"<text>" << title << L"</text>" <<
      L"<text>" << content << L"</text>" <<
      L"<image src='" << image << L"'/>" <<
      L"<image src='" << logo << L"'" <<
      L" placement='appLogoOverride' hint-crop='circle'/>" <<
      L"</binding></visual></toast>";    XmlDocument toastXml;    toastXml.LoadXml(xmlString.str().c_str());

   ToastNotificationManager::CreateToastNotifier().Show(ToastNotification(toastXml)); }
```

```
using namespace Windows::Foundation; using namespace
Windows::System;
using namespace Windows::UI::Notifications; using namespace
Windows::Data::Xml::Dom;

void UWP::ShowToast()
{
 Platform::String ^title = "featured picture of the day";
 Platform::String ^content = "beautiful scenery";
 Platform::String ^image = "https://picsum.photos/360/180?image=104";  Platform::String ^logo =
"https://picsum.photos/64?image=883";

 Platform::String ^xmlString =
  L"<toast><visual><binding template='ToastGeneric'>" +
  L"<text>" + title + "</text>" +
  L"<text>"+ content + "</text>" +
  L"<image src='" + image + "'/>" +
  L"<image src='" + logo + "'" +
  L" placement='appLogoOverride' hint-crop='circle'/>" +
  L"</binding></visual></toast>";

 XmlDocument ^toastXml = ref new XmlDocument();  toastXml->LoadXml(xmlString);

 ToastNotificationManager::CreateToastNotifier()->Show(ref new ToastNotification(toastXml));
}
```

알림에 대하여 더 많은 정보를 알고 싶다면 적응적, 상호작용 토스트 알림을 참조하십시오.

Windows XP, Vista, 7/8 설치 기반 지원하기

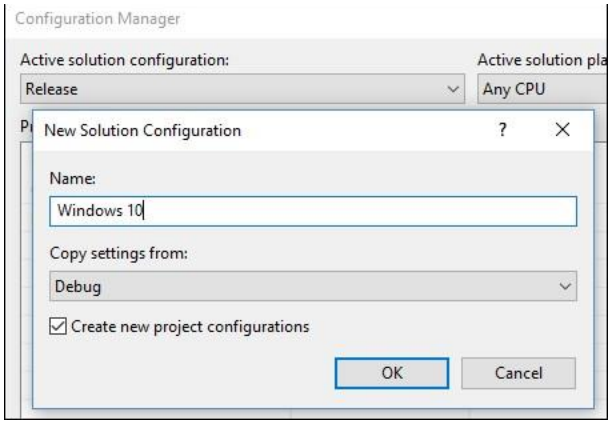
새 분기를 생성하고 별도의 코드베이스를 유지 관리할 필요 없이 Windows 10 용 응용 프로그램을 현대화할 수 있습니다.

Windows 10 사용자를 위한 별도의 이진 파일을 작성하려면 조건부 컴파일을 사용합니다. 모든 Windows 사용자에게 배포하는 이진 파일 집합을 하나 작성하려면 런타임 검사를 사용하십시오.

각 옵션에 대하여 짧게 알아보겠습니다.

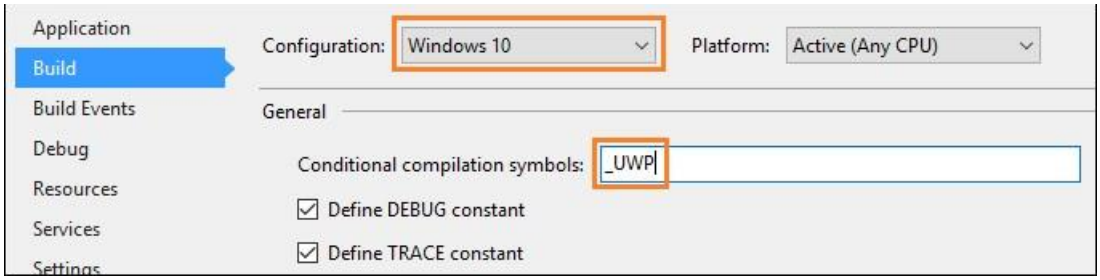
조건부 컴파일

하나의 코드베이스를 유지하고 Windows 10 사용자만을 위한 이진 파일 집합을 컴파일할 수 있습니다. 먼저 새 빌드 구성을 프로젝트에 추가합니다.

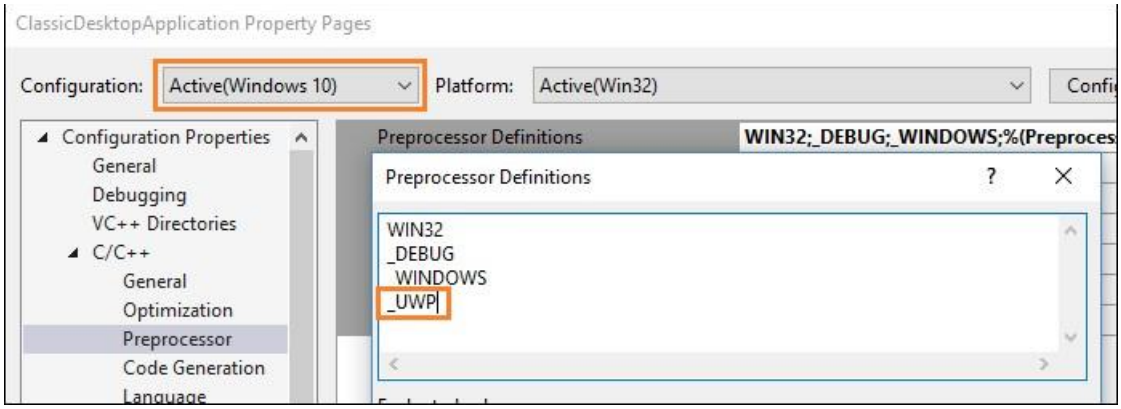


해당 빌드 구성의 경우 Windows 런타임 API 를 호출하는 코드를 식별할 상수를 생성합니다.

.NET 기반 프로젝트의 경우 이 상수는 조건부 컴파일 상수라고 합니다.



C++ 기반 프로젝트에서는 이 상수를 전처리기 정의라고 부릅니다.



UWP 코드 블록 앞에 해당 상수를 추가합니다.

```
[System.Diagnostics.Conditional("_UWP")] private void
ShowToast() { ... }
```

```
#if _UWP
void UWP::ShowToast() { ... }
#endif
```

컴파일러는 해당 상수가 활성 빌드 구성에서 정의된 경우에만 해당 코드를 빌드합니다.

런타임 체크

실행하는 Windows 버전에 관계없이 모든 Windows 사용자에 대해 하나의 이진 파일을 컴파일할 수 있습니다. 응용 프로그램은 사용자가 응용 프로그램을 Windows 10 에서 패키지 응용 프로그램으로 실행하는 경우에만 Windows 런타임 API 를 호출합니다.

런타임 체크를 추가하는 가장 쉬운 방법은 데스크톱 브리지 헬퍼 Nuget 패키지를 설치하고 IsRunningAsUWP() 함수를 사용하여 Windows 런타임 API 를 호출하는 모든 코드의 게이트를 해제하는 것입니다. 더 자세한 사항은 다음 블로그 포스트를 참조하십시오. 데스크톱 브리지 – 애플리케이션 컨텍스트 식별

관련 샘플

- Hello World 샘플
- 보조 타일 샘플
- 스토어 API 샘플
- UWP UpdateTask 를 구현하는 WinForms 앱
- Desktop app bridge to UWP 샘플

질문에 대한 답을 찾아보기

질문이 있으신가요? Stack Overflow 에서 Microsoft 에 질문하세요. 이 팀은 이 태그들을 지속 확인합니다. MSDN 포럼에서도 질문할 수 있습니다.